Godot CPP 3D Tutorial [Work in progress]

By Ken Burchfiel

This step-by-step guide will demonstrate how to create a 3D multiplayer game in Godot using C++. It is based on my Cube Combat demo, but (unlike that project) guides you more explicitly through the steps involved, both within your code editor and Godot, to put this kind of game together.

Note: This resource is being created without the use of generative-AI tools.

Part 1: Getting started

(Note: Many of these steps will resemble those in the excellent 'Getting started' section of the official GDExtension documentation at https://docs.godotengine.org/en/4.6/tutorials/scripting/cpp/gdextension_cpp_example.html . Certain code blocks within this section derive from that document as well.)

1. Create a new folder that will store your Godot project and its corresponding code. I'll call mine `godot_cpp_3d_tutorial`, but the name you choose is of course up to you.

2. First, you'll want to download the latest stable version of godot-cpp from https://github.com/godotengine/godot-cpp . (As of 2026-04-17, a stable version of version 10.x hasn't yet been released, so I went ahead and downloaded the beta version with a commit ID of 4862a9d (https://github.com/godotengine/godot-cpp/tree/4862a9dcf1471c9ea19680b9faadb5b6a9432092 .) Whether you download and unzip or simply clone it, make sure that exists within your project folder within a folder named 'godot-cpp'.

3. Next, open up this godot-cpp folder within your terminal and run `scons platform=linux` (replacing `linux` with your own OS if needed). This will compile all of the source code needed to apply this library. (It will *also* generate additional code files that aren't visible in an uncompiled version of the repository, such as the one on GitHub).

4. Go ahead and create a 'src' folder within your root project folder. This folder will store your source C++ code and its compiled variants.

5. Create a 'project' folder within this root folder also. Next, open up Godot, which you can download from https://godotengine.org/ if you haven't already. (I'm using Godot 4.6 for this project, but this tutorial should be applicable for newer releases also for a decent while.) Within the loading screen, hit the Create button at the top left. I chose 'Cpp 3D Tutorial' as my project name and the 'project' folder I just created as my path. Once you've filled in these items, hit Create on the bottom right.

   ![](## Godot CPP 3D Tutorial [Work in progress]

By Ken Burchfiel

This step-by-step guide will demonstrate how to create a 3D multiplayer game in Godot using C++. It is based on my Cube Combat demo, but (unlike that project) guides you more explicitly through the steps involved, both within your code editor and Godot, to put this kind of game together.

Note: This resource is being created without the use of generative-AI tools.

Part 1: Getting started

(Note: Many of these steps will resemble those in the excellent 'Getting started' section of the official GDExtension documentation at https://docs.godotengine.org/en/4.6/tutorials/scripting/cpp/gdextension_cpp_example.html . Certain code blocks within this section derive from that document as well.)

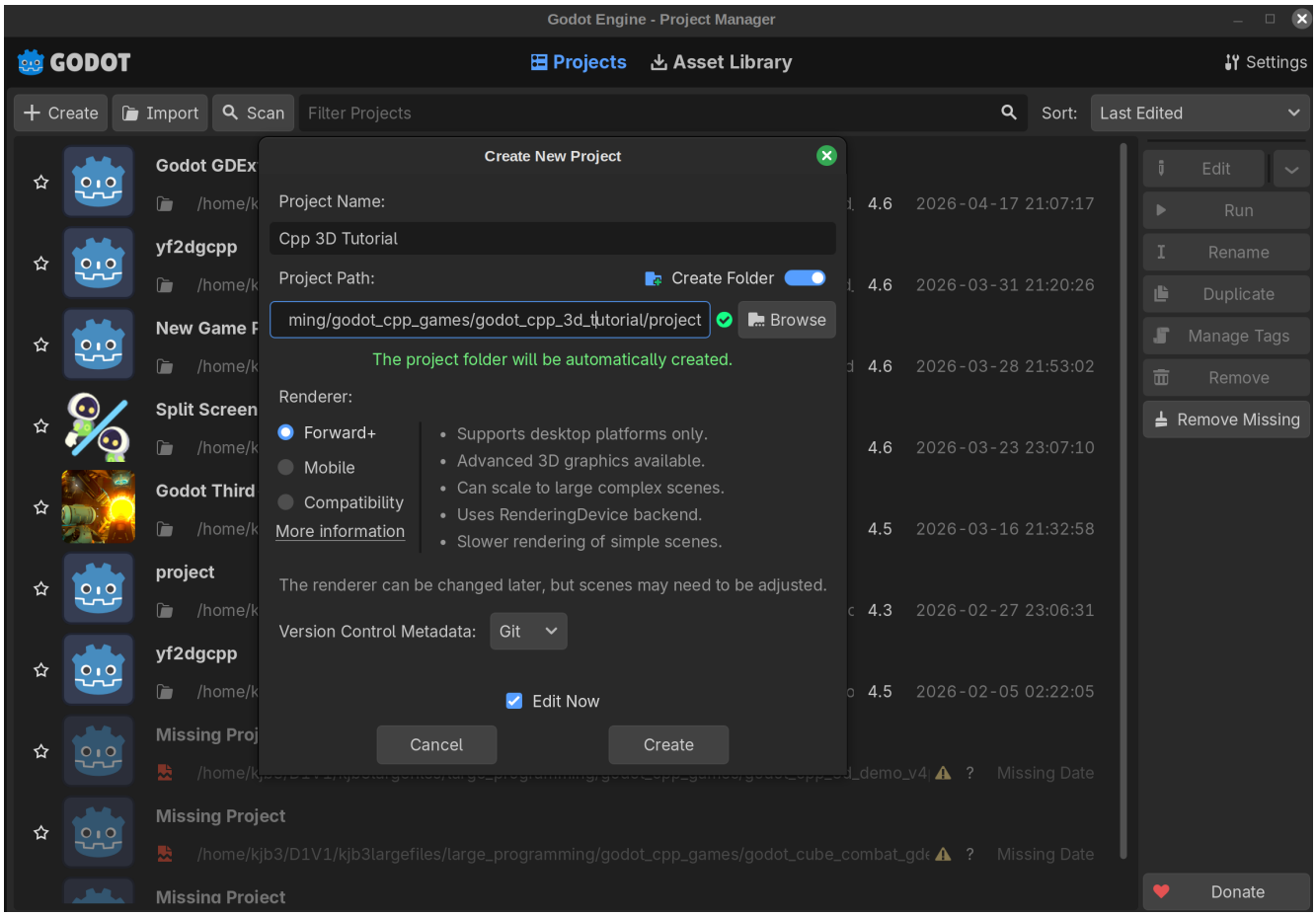
1. Create a new folder that will store your Godot project and its corresponding code. I'll call mine `godot_cpp_3d_tutorial`, but the name you choose is of course up to you.

2. First, you'll want to download the latest stable version of godot-cpp from https://github.com/godotengine/godot-cpp . (As of 2026-04-17, a stable version of version 10.x hasn't yet been released, so I went ahead and downloaded the beta version with a commit ID of 4862a9d (https://github.com/godotengine/godot-cpp/tree/4862a9dcf1471c9ea19680b9faadb5b6a9432092 .) Whether you download and unzip or simply clone it, make sure that exists within your project folder within a folder named 'godot-cpp'.

3. Next, open up this godot-cpp folder within your terminal and run `scons platform=linux` (replacing `linux` with your own OS if needed). This will compile all of the source code needed to apply this library. (It will *also* generate additional code files that aren't visible in an uncompiled version of the repository, such as the one on GitHub).

4. Go ahead and create a 'src' folder within your root project folder. This folder will store your source C++ code and its compiled variants.

5. Create a 'project' folder within this root folder also. Next, open up Godot, which you can download from https://godotengine.org/ if you haven't already. (I'm using Godot 4.6 for this project, but this tutorial should be applicable for newer releases also for a decent while.) Within the loading screen, hit the Create button at the top left. I chose 'Cpp 3D Tutorial' as my project name and the 'project' folder I just created as my path. Once you've filled in these items, hit Create on the bottom right.



6. Close back out of the editor for now. Before we create a scene, we should first create a GDExtension class within C++ that can be used as the basis for that scene. (This is a different approach than what you might be used to with GDScript.)

7. Before we start writing our own C++ code, let's get a few crucial setup tasks out of the way. First, within your project folder, which will now have a number of Godot-generated items (including a project.godot) file, go ahead and create a new folder called 'bin.' Within this folder, create a new file called gdexample.gdextension, then paste the following material into it:

```
[configuration]

entry_symbol = "example_library_init"
compatibility_minimum = "4.1"
reloadable = true

[libraries]

macos.debug = "./libgdexample.macos.template_debug.dylib"
macos.release = "./libgdexample.macos.template_release.dylib"
windows.debug.x86_32 = "./gdexample.windows.template_debug.x86_32.dll"
windows.release.x86_32 =
"./gdexample.windows.template_release.x86_32.dll"
windows.debug.x86_64 = "./gdexample.windows.template_debug.x86_64.dll"
```

```
windows.release.x86_64 =
"./gdexample.windows.template_release.x86_64.dll"
linux.debug.x86_64 = "./libgdexample.linux.template_debug.x86_64.so"
linux.release.x86_64 = "./libgdexample.linux.template_release.x86_64.so"
linux.debug.arm64 = "./libgdexample.linux.template_debug.arm64.so"
linux.release.arm64 = "./libgdexample.linux.template_release.arm64.so"
linux.debug.rv64 = "./libgdexample.linux.template_debug.rv64.so"
linux.release.rv64 = "./libgdexample.linux.template_release.rv64.so"
```

(This is an exact copy of the corresponding code within Godot's official GDExtension documentation (Reference 1)).

(I could have changed 'gdexample' and 'example_libary_init' to more meaningful entries, but for boilerplate like this, it's often safest to stick with the existing version.)

8. Next, within your src folder, create a new file called 'register_types.h.' Copy and paste the following code into that file:

```
#pragma once

#include <godot_cpp/core/class_db.hpp>

using namespace godot;

void initialize_example_module(ModuleInitializationLevel p_level);
void uninitialize_example_module(ModuleInitializationLevel p_level);
```

(Source: Reference 1)

9. These two files won't need to be modified further. The same is not the case for the following code, which we'll update over time to include the classes that we'll create within this tutorial. Go ahead and paste it into a new 'register_types.cpp' file within your src folder:

```
#include "register_types.h"

#include <gdextension_interface.h>
#include <godot_cpp/core/defs.hpp>
#include <godot_cpp/godot.hpp>

using namespace godot;

void initialize_example_module(ModuleInitializationLevel p_level) {
    if (p_level != MODULE_INITIALIZATION_LEVEL_SCENE) {
        return;
```

```
    }

}

void uninitialize_example_module(ModuleInitializationLevel p_level) {
    if (p_level != MODULE_INITIALIZATION_LEVEL_SCENE) {
        return;
    }
}

extern "C" {
// Initialization.
GDExtensionBool GDE_EXPORT
example_library_init(GDExtensionInterfaceGetProcAddress
p_get_proc_address, const GDExtensionClassLibraryPtr p_library,
GDExtensionInitialization *r_initialization) {
    godot::GDExtensionBinding::InitObject init_obj(p_get_proc_address,
p_library, r_initialization);

    init_obj.register_initializer(initialize_example_module);
    init_obj.register_terminator(uninitialize_example_module);

init_obj.set_minimum_library_initialization_level(MODULE_INITIALIZATION_L
EVEL_SCENE);

    return init_obj.init();
}
}
```

(Source: Reference 1, with a few lines removed so that we can add in our own versions later)

10. We'll also need to add a SConstruct file to our root folder in order to successfully compile our GDExtension classes. Visit https://docs.godotengine.org/en/4.6/_downloads/8df537a8866633825684515bce40faa6/SConstruct , which should prompt you to save an SConstruct file to your computer. Go ahead and save it into your root folder as 'SConstruct' (without any extension). (You can also access this download by visiting https://docs.godotengine.org/en/4.6/tutorials/scripting/cpp/gdextension_cpp_example.html , then searching for a link with the text "the SConstruct file we prepared.")

    Here's what your root folder should look like at this point (excluding certain Godot-created project files):

```
godot_cpp_3d_tutorial/ [my root folder name; your name may vary]

--godot-cpp/ [contains your compiled godot-cpp repository]

--project/
```

```
----bin/

------gdexample.gdextension

--src/

----register_types.cpp/

----register_types.h

--SConstruct
```

11. Although we haven't created any classes yet, this will be a good time to compile the code we've added in so far. Navigate to your root folder and then run `scons platform=[your_os]` (e.g. `scons platform=linux` in my case). This is the same command you used to compile godot-cpp--just in your project folder rather than the godot-cpp folder. You should see `scons: done building targets.` appear in the terminal after all files have been compiled and linked.

Part 2: Adding in a class

Now that we've gotten those setup tasks out of the way, we can begin programming our own GDExtension classes. Let's start with the Main class, which will govern the game area and some fundamental gameplay logic.

1. Within your src folder, create a new file called 'main.h'. Copy the following code into this file:

```
#pragma once
#include <godot_cpp/classes/node.hpp>
#include <godot_cpp/variant/utility_functions.hpp>

using namespace godot;

class Main: public Node {GDCLASS(Main, Node)

protected:

static void _bind_methods();

public:
Main();
~Main();

void _ready();

};
```

(Source: Reference 8)

Here, we're declaring a GDExtension class called 'Main' that will inherit from Node. We're also adding some fundamental functions (e.g. constructors, destructors, _bind_methods(), and _ready()). Lots more will be added to this file later on, but I wanted to focus on the items we'll need at this moment.

2. Next, create a file within src/ called 'main.cpp' and paste the following code into it:

```
#include "main.h"

void Main::_bind_methods() {}

Main::Main() {}

Main::~Main() {}

void Main::_ready() {

    UtilityFunctions::print("Main::_ready() just got called.");
}
```

(Source: Reference 8)

This is all pretty barebones so far, but we'll expand this file quite a bit later in the tutorial.

We don't need to add anything just yet to `_bind_methods()`, but since that function is a core part of many GDExtension classes, I figured I would add it here.

3. Go back to register_types.cpp. Directly under `#include "register_types.h"`, add:

```
#include "main.h"
```

4. Next, within the `initialize_example_module()` function, add the following text right before the final closing bracket:

```
GDREGISTER_RUNTIME_CLASS(Main);
```

These two updates allow Godot to learn about our Main GDExtension class. We'll repeat these simple updates for all other classes that we define.

Note: I'm using GDREGISTER_RUNTIME_CLASS here rather than GDREGISTER_CLASS so that this code will only run when I'm actually running my project. With GDREGISTER_CLASS, code can also run (or at least attempt to run) in the editor, which can cause some irritating crashes. (For instance, suppose your code

for a class references a Pivot object that you haven't yet created. If you then attempt to launch the editor after compiling this code, the editor will attempt to find this object, fail, and potentially crash. To avoid the need to comment out that code, add the object into your editor, and then recompile it, you can simply make that class a *runtime* class.)

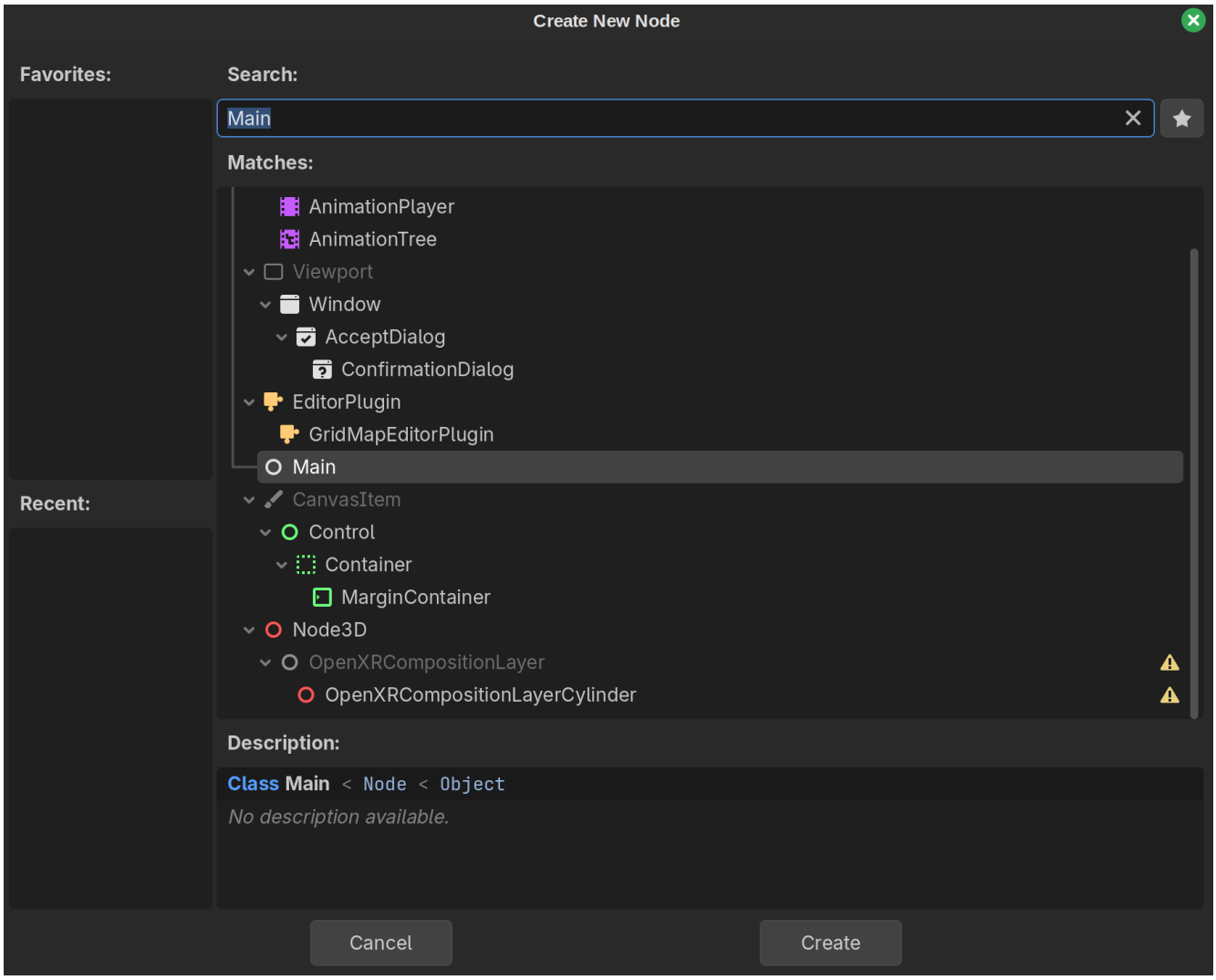
(Sources: References 8, 9, and 10)

5. Go ahead and run `scons platform=[your_os]` again. (If you haven't closed your terminal since the last time you ran this command, you may be able to access this line by pressing your Up Arrow key.) You should again see `scons: done building targets.` once the compilation process completes.

Part 3: Setting up main.tscn

Now that we have a class (albeit a very simple one), we can create a scene based on it.

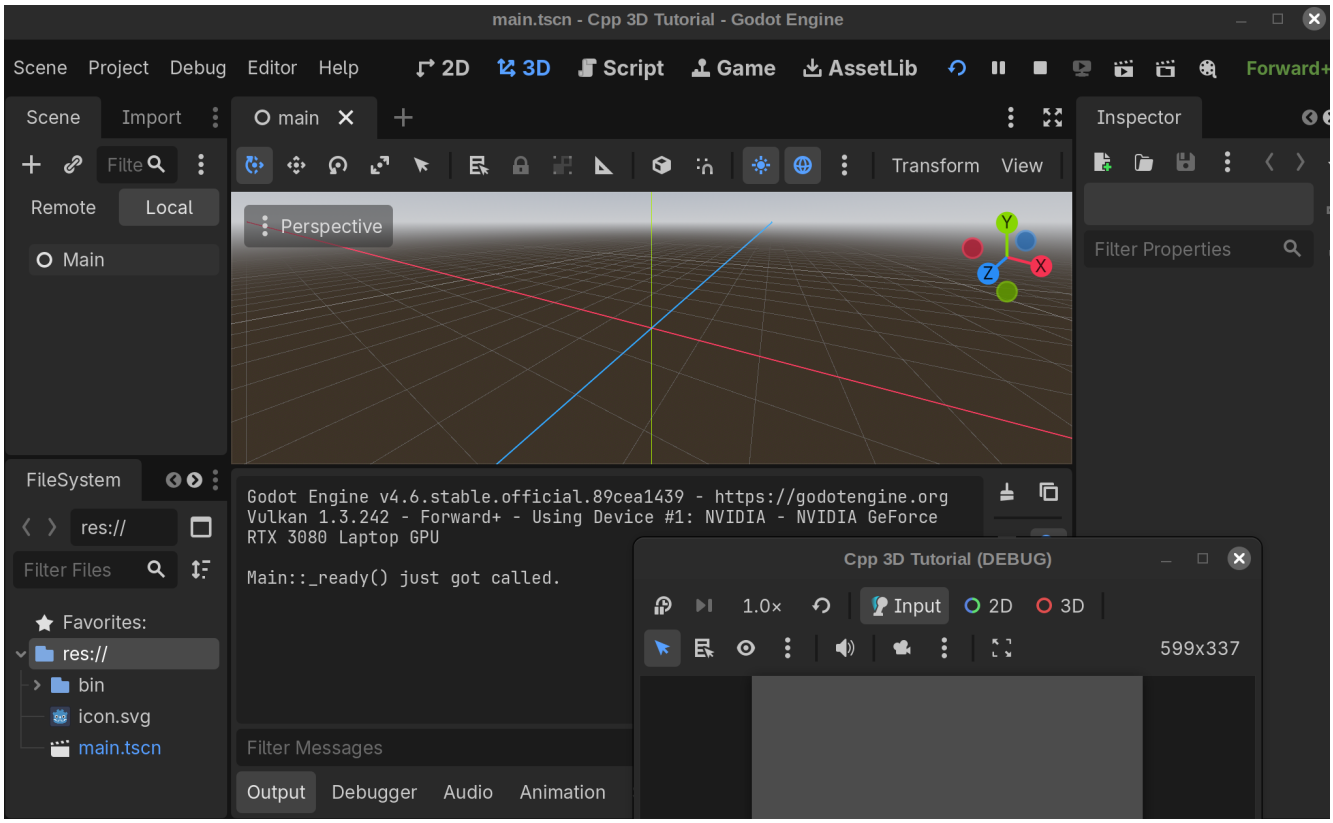
1. Reopen Godot and open your project. Next, within the 'Create Root Node:' menu, select 'Other Node' and enter 'Main' in the search bar. Hopefully, you will see your newly-created Main node within the list of available nodes:



If you *don't* see this node (a situation I've faced quite a few times), this likely means that you forgot one of the earlier steps (such as adding references to this node to your register_types.cpp file).

2. Select this node, then save your scene as main.tscn. Next, click the play button near the top right of the editor in order to get the debug message we defined within `Main::_ready()` to display. This will bring up a scene-selection dialog box; you can hit 'Select Current', since we'll indeed want this to be our main scene.

3. A gameplay window with a gray box will open. Nothing will appear inside it, which is to be expected. However, you *should* see `Main::_ready() just got called.` appear within your Output tab in the lower half of the main editor window:



4. While we're inside the editor, let's go ahead and create a game area. (Many of the following steps were based on the 'Setting up the game area' section of the Your First 3D Game Tutorial (Reference 3).) Right click on Main and select Add Child Node, then search for (and select) StaticBody3D. Rename it Ground within your scene tree on the left side of the editor. Next, add a CollisionShape3D and a MeshInstance3D as children of Ground.

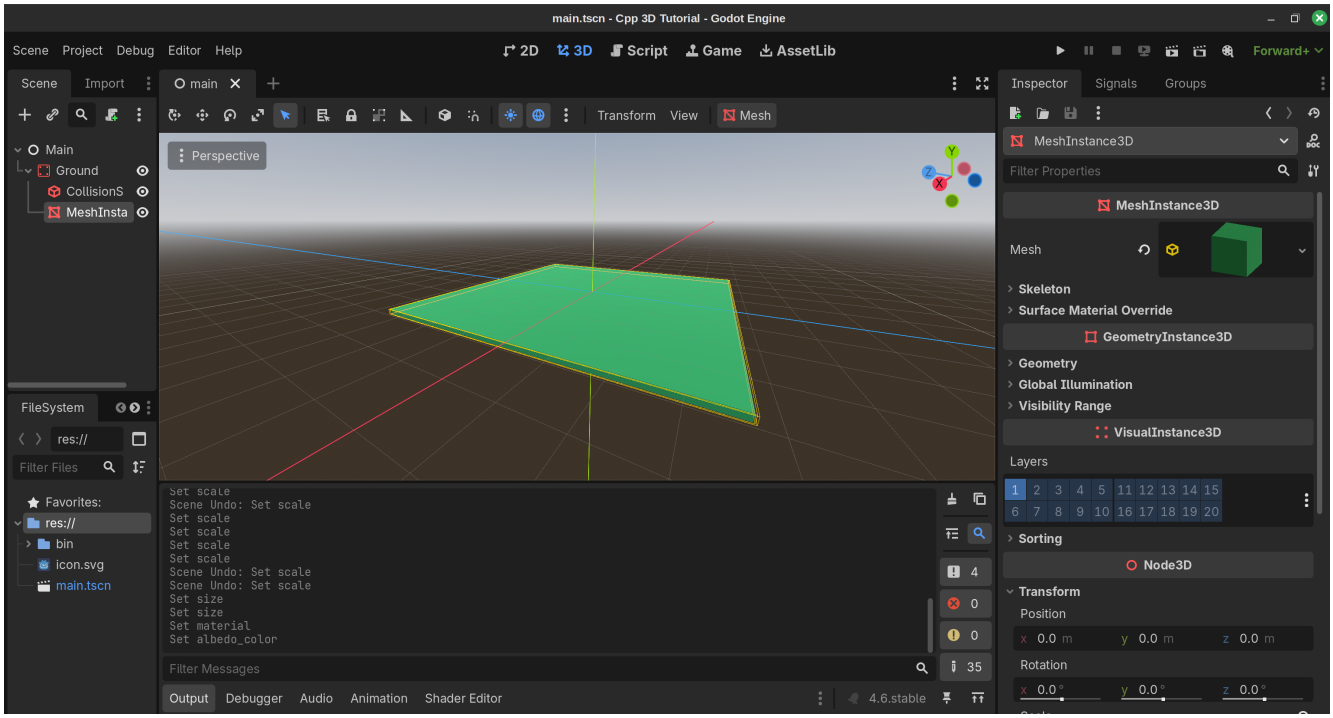
5. Select the MeshInstance3D item within the scene tree if you haven't already. Click the 'empty' box to the right of Mesh in the Inspector and select a new BoxMesh. Next, click on CollisionShape3D within the scene tree; within the Shape row of the Inspector, choose a BoxShape3D.

6. Next, click on the Ground object within the scene tree, then click on Transform within the Inspector. Change the y entry within the Position (not Scale!) menu to -0.5 so that the top of the Ground, which is 1 meter thick, has a y position of 0 rather than 0.5.

7. Click the CollisionShape resource, then select the BoxShape. Within the Size menu that appears, change the x and z values to 60; keep the y value untouched at 1. Next, click on the MeshInstance, then click on the chain to the right of 'Scale' within the Transform section such that it appears broken. (This way, changes to one dimension won't affect the others.) As you did with the BoxShape, change the x and z scales to 60.0 and leave the y scale untouched at 1.0. Your Ground object should now look like a thin square.

8. Let's change the drab, white color of the Ground to something more interesting. Select your MeshInstance3D in the scene tree, then click the downwards-facing arrow to the right of the Mesh (and its corresponding gray cube) in the Inspector and select 'Edit.' Within the Material section, create a new StandardMaterial3D, then click the downwards arrow to the right of the white sphere that appears and select Edit. Click on the Albedo section, then select a color of your choice.

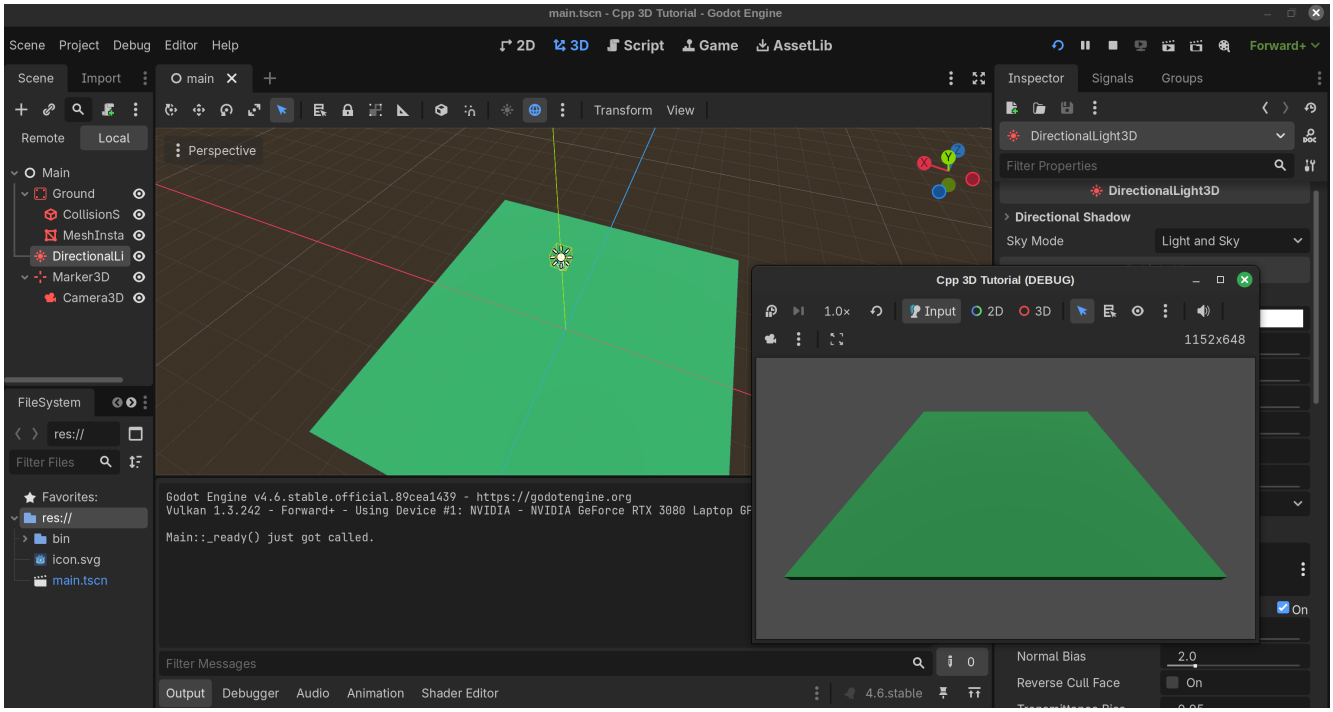
   Here's what the Ground should look like at this point:



9. Before we can get to the 'action' part of this scene, we'll need lights and a camera. Add a DirectionalLight3D as a child of Main, then set its y transform to 20.0. (The x and z transforms can stay at 0.0.) Change its x rotation to -90, or whatever allows the light to point directly down at the ground. (You'll know it's working when you see the ground brighten up.) Check the Shadow box within the Light3D section of the Inspector as well.

10. Next, add a Marker3D as a child of Main and set its y and z transforms to 25.0 and -40.0, respectively. Finally, add a Camera3D as a child of the Marker3D. Set its x and y rotations (accessible within the Transform section of the Inspector) to -45 and 180, respectively. (You can scroll up to the top of the Inspector to get a preview of what the camera will display.)

11. Try running the scene again. You should now see your game area within the window that appears:



12. Now that we have a game scene in place, this will be a good time to begin work on our Mnchar (main character) class. There's plenty more C++ code that will get added to main.cpp and main.h, but those additions will be easier to implement and debug once we have actual characters and projectiles to manage.

Part 4: Laying the foundations for our Mnchar class

Our Mnchar class, which players will be able to control via game controllers, will fire projectiles and (potentially) get hit by other projectiles. We'll eventually configure our game such that anywhere from 2-8 Mnchars can get added to the game scene at the start of each game; however, that configuration will involve a Hud class that we won't be setting up for a little while.

1. To begin the setup process, create both a 'mnchar.h' and a 'mnchar.cpp' file within your src folder. Enter the following code into mnchar.h:

```
#pragma once

#include <godot_cpp/classes/character_body3d.hpp>
#include <godot_cpp/variant/utility_functions.hpp>

using namespace godot;

class Mnchar : public CharacterBody3D {
GDCLASS(Mnchar, CharacterBody3D)

private:
double movement_speed = 14;
double rotation_speed = 0.15;
```

```
protected:
static void _bind_methods();

public:
Mnchar();
~Mnchar();

void set_movement_speed(const double movement_speed);
double get_movement_speed() const;

void set_rotation_speed(const double rotation_speed);
double get_rotation_speed() const;


};
```

(Source: References 1, 2, and 4)

This code is very similar to that found within main.h. Two new additions of note are `movement_speed` and `rotation_speed` along with their corresponding setter and getter functions. We'll make it possible to update both of these values within the editor.

Also note that, because Mnchar will extend CharacterBody3D, we need to include its header file within this source file. (I chose to use CharacterBody3D as my player's class because the Your First 3D Game tutorial uses this same class; see Reference 5. Code for the CharacterBody3D class itself can be found in References 6 and 7.

2. Next, within mnchar.cpp, enter the following:

```
#include "mnchar.h"
#include <godot_cpp/core/class_db.hpp>

using namespace godot;

void Mnchar::_bind_methods() {
ClassDB::bind_method(D_METHOD("get_movement_speed"),
                     &Mnchar::get_movement_speed);
ClassDB::bind_method(D_METHOD("set_movement_speed", "p_movement_speed"),
                     &Mnchar::set_movement_speed);
ADD_PROPERTY(PropertyInfo(Variant::FLOAT, "movement_speed"),
            "set_movement_speed", "get_movement_speed");

ClassDB::bind_method(D_METHOD("get_rotation_speed"),
                     &Mnchar::get_rotation_speed);
ClassDB::bind_method(D_METHOD("set_rotation_speed", "p_rotation_speed"),
                     &Mnchar::set_rotation_speed);
```

```
ADD_PROPERTY(PropertyInfo(Variant::FLOAT, "rotation_speed"),
            "set_rotation_speed", "get_rotation_speed");

}

Mnchar::Mnchar() {}

Mnchar::~Mnchar() {}



void Mnchar::set_movement_speed(const double p_movement_speed) {
movement_speed = p_movement_speed;
}

double Mnchar::get_movement_speed() const { return movement_speed; }

void Mnchar::set_rotation_speed(const double p_rotation_speed) {
rotation_speed = p_rotation_speed;
}

double Mnchar::get_rotation_speed() const { return rotation_speed; }
```

(Source: References 1 and 2)

This code, like that in main.h, is quite barebones; our priority is simply to add in a class that the Godot editor will recognize. Once we create and configure a Mnchar scene within the editor, we'll then come back and extend this code.

However, I did also add in code that will allow us to modify the movement and rotation speeds within the editor. This isn't a crucial part of this particular game, but it *will* allow you to test out different movement and rotation values without having to recompile the code--not that that's a huge hurdle.

The speed-adjustment code consists of two functions (`set_movement_speed()` and `get_movement_speed()`) that let us specify and retrieve, respectively, the player's speed. Both of these functions (including the former's `p_movement_speed` argument) have corresponding bind_method() calls within `_bind_methods()`. In addition, we have an ADD_PROPERTY() call that tells the editor about the `movement_speed` variable along with its setter and getter functions. (These are all based on the `amplitude` property within the GDExample class in Reference 1.)

The rotation-adjustment code is very similar. I simply copied and pasted my relevant movement-adjustment code, then replaced all cases of 'movement' with 'rotation.'

(It is *not* necessary to add in `bind_method()` and `ADD_PROPERTY()` calls like this for every attribute of a class. However, they're very important for certain items, such as signals--which we'll get to later.)

3. This new code should compile at this point; however, if we tried to do so, we most likely wouldn't be able to locate a Mnchar class within our editor. That's because we also need to update register_types.cpp with

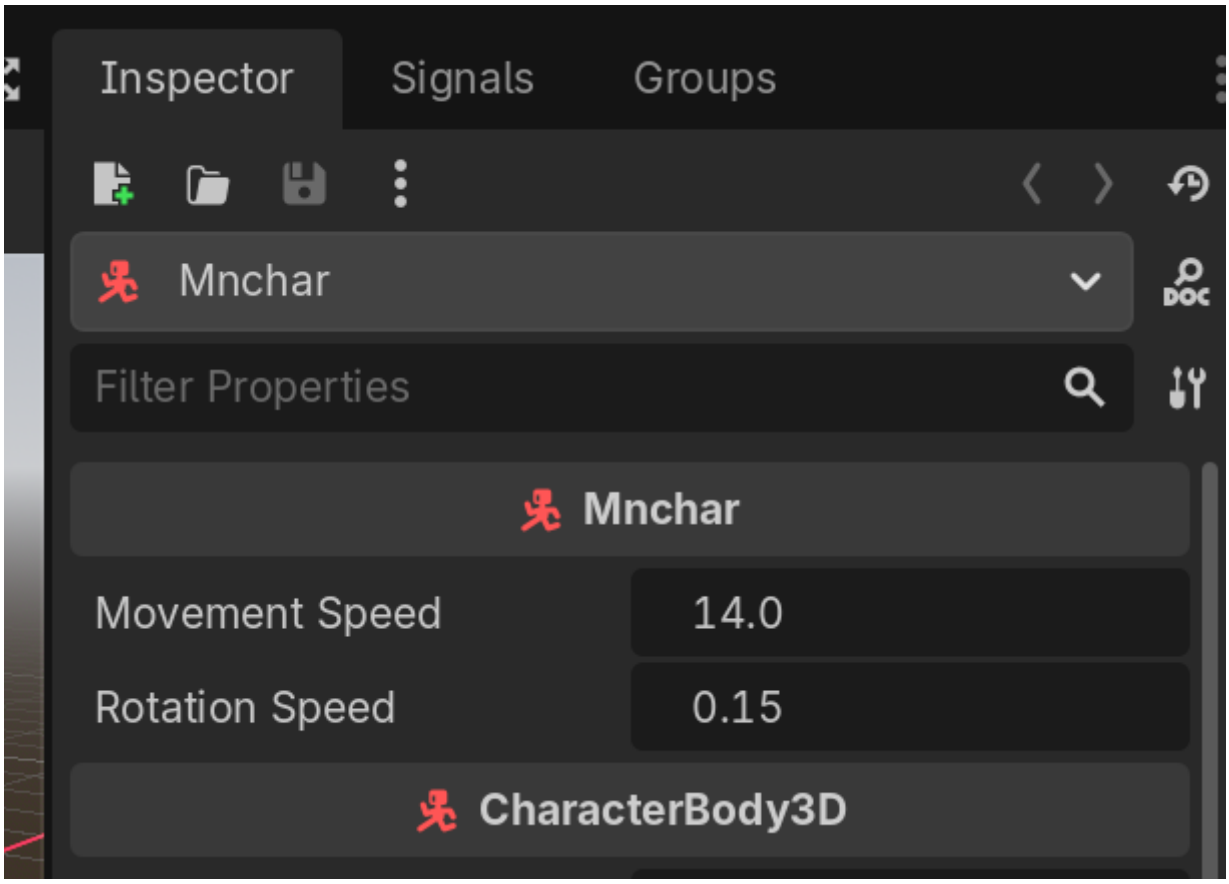
information about this class. Fortunately, this is easy to do so. Right under `#include "main.h"`, add `#include "mnchar.h"`. Next, right under `GDREGISTER_RUNTIME_CLASS(Main);`, add `GDREGISTER_RUNTIME_CLASS(Mnchar);`.

4. *Now* run `scons platform=[your_os]` to compile this new Mnchar-related code. In my case, the editor still didn't show the Mnchar class within the 'Create New Node' menu after this step, but it did present it once I closed and relaunched my editor. Thus, I'd recommend that you do the same at this point.

Part 5: Setting up mnchar.tscn

1. Back in the editor, create a new scene. Click the 'Other Node' button under the 'Create Root Node:' prompt; search for 'Mnchar'; then double-click it. Next, save this scene as mnchar.tscn.

   You *should* see the custom Movement Speed property that we configured near the top of the Inspector menu, along with our default value (14). Changing this value in the editor will also change the Mnchar's behavior within our game.



   (I have found, however, that this property will sometimes disappear from the editor after compiling my code. This might be caused by an issue with my current setup, but it might also be a glitch within the editor itself. Closing, then relaunching the editor always seems to resolve this issue, thankfully.)

2. Add a Node3D as a child of Mnchar and rename it 'Pivot'. In many cases, we'll apply movement and rotation actions to this node rather than to Mnchar itself. Next, add a MeshInstance as a child of Pivot; name it 'Body'; click the 'empty' text within its Mesh section in the Inspector; and select a BoxMesh. Next,

click the downwards-pointing arrow to the right of the gray cube in the Inspector and select Edit. (Reference 5)

3. Within the Size section of the edit menu, change the x, y, and z values from 1.0 to 2.0. Next, go down to the Material section and select a new StandardMaterial 3D. Unlike with the game area, we won't choose a color for this material just yet--as we'll actually use C++ to update this color instead. (This will make it easier to assign different colors to different Mnchar instances.)

4. We'll also want to create a turret for our player. Create a new MeshInstance3D child of Pivot; name it 'Turret'; assign it a new BoxMesh; go into that mesh's edit menu; change the x, y, and z sizes to 0.5, 0.5, and 0.5; and give it a new StandardMaterial3D.

5. Next, navigate back to the Turret's main Inspector menu. (You can do so a few different ways; one of which is to select the Body, then the Turret again.) Within the Node3D section, set the z transform to 1.25 meters. That way, the turret will be adjacent to the forward face of the Pivot.

6. Add a CollisionShape3D as a child of *Mnchar* (not Pivot). Click on the 'empty' text within the Shape section of the Inspector; add in a BoxShape3D; click the downwards arrow to the right of 'BoxShape3D'; and select 'Edit.' Change the x, y, and z Size values to 2.0 in order to make it the exact same size as the Body component.

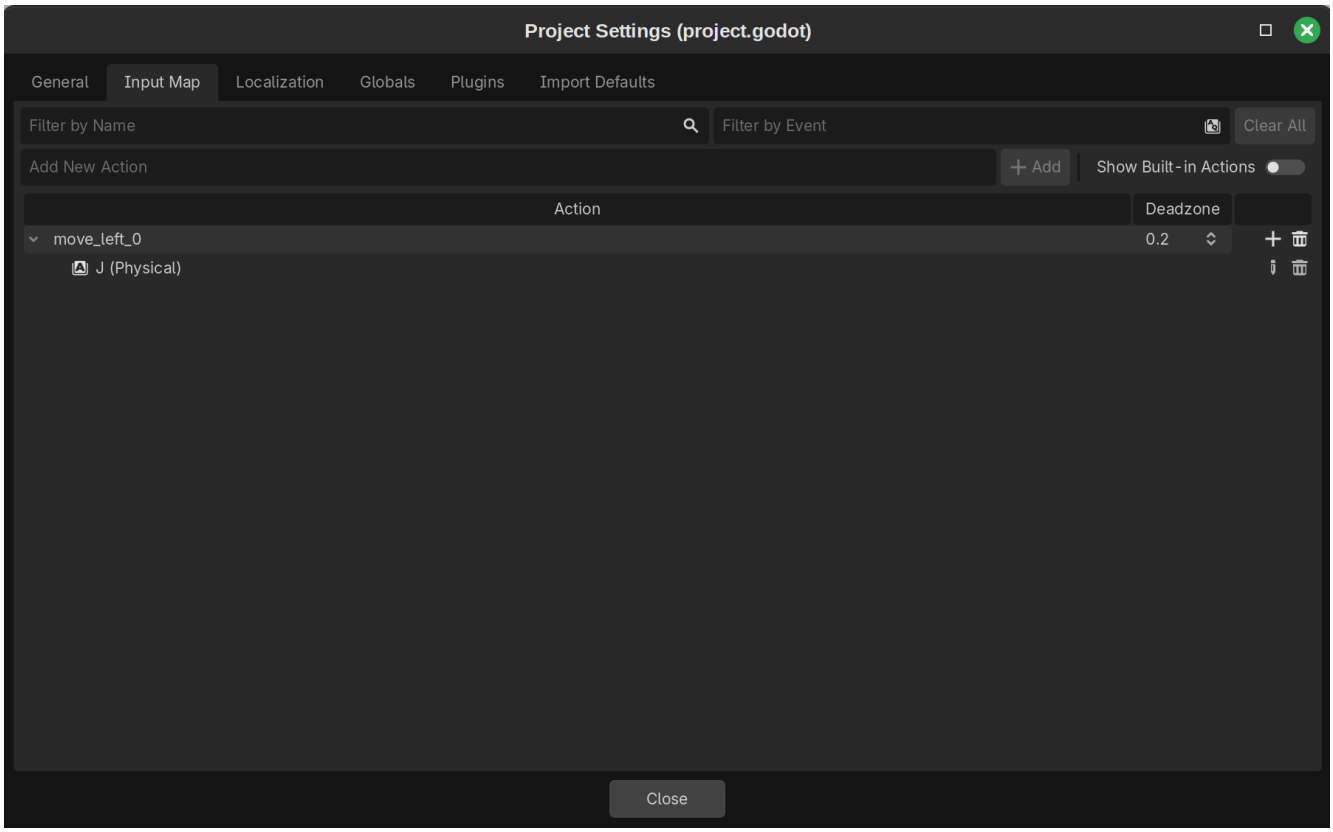
   (You could also update these Size values, along with the transform of the CollisionShape, to allow it to fit over both the Body and Turret objects; alternatively, you could create a separate CollisionShape3D for the Turret. That would prevent the Turret from being able to enter into other objects. But this simpler approach will suffice for this tutorial.) (Reference 5)

   Once you're finished with these updates, go ahead and save the scene.

7. In order to move the Mnchar with a controller (or, for development purposes, a keyboard), we'll need to add input actions to our game's Input Map. Navigate to this map by clicking Project in the top left of the Godot editor window, then selecting Project Settings; the Input Map should be the second tab from the left. (Reference 5)

8. For now, we'll just add in keyboard entries; once we're further in the development process, we'll add in controller entries also. Click on the 'Add New Action' text within the Input Map menu; type move_left_0; and hit the '+ Add' button to the right of this window. Next, click the + sign to the right of the new 'move_left_0' entry that has appeared; and hit your J key (or, if you're using a different layout like I am, where the J key would be on a QWERTY keyboard).

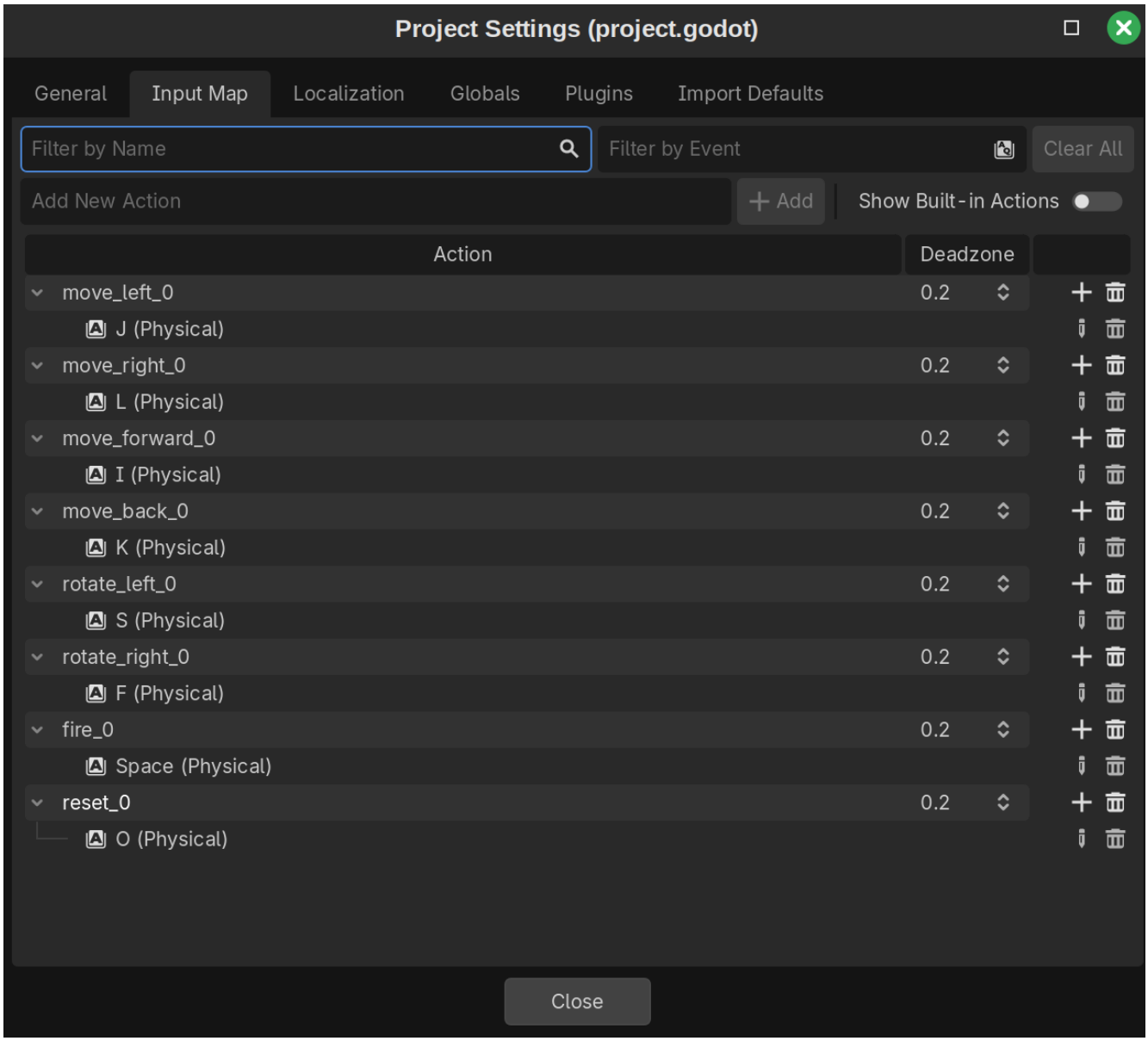
   (You're welcome to use a key other than J, such as Left Arrow, if you'd like. The use of 'J' will make more sense in the context of all the keys we'll be adding in.)



9. Perform the same steps for the following action names and keys:

    1. move_right_0 (L key)
    2. move_forward_0 (I key)
    3. move_back_0 (K key)
    4. rotate_left_0 (S key)
    5. rotate_right_0 (F key)
    6. fire_0 (Space Bar)
    7. reset_0 (O key)

10. Once you've finished this process, your input map should look like the following:



By the way, the reason for adding '_0' to the end of these actions is to allow different sets of controls to be distinguished for different players later on.

Close out of the input map and save your mnchar.tscn file.

Part 6: Adding a Mnchar to the game area

We're almost ready to add in code that will let us move our Mnchar around the game area. First, though, we need to *add* the Mnchar to the game area.

One option would be to instantiate a Mnchar as a child scene of main.tscn (Reference 4). However, since we're creating a multiplayer game whose player count might change from round to round, it will be more ideal to add Mnchars to this scene via code. (That way, a specific number of Mnchars can be placed within the game area depending on how many players choose to enter a given game.) The following steps will allow us to add a Mnchar to the scene using C++.

1. Within main.h, add the following code right above `protected:`

```
private:
    Ref<PackedScene> mnchar_scene;
```

Next, add the following right below `~Main();`

```
Ref<PackedScene> get_mnchar_scene();
void set_mnchar_scene(Ref<PackedScene>);
```

(Reference 8)

Finally, after `#include <godot_cpp/variant/utility_functions.hpp>` , add:

```
#include <godot_cpp/classes/packed_scene.hpp>
#include "mnchar.h"
```

2. Next, within main.cpp, add the following code within `Main::_bind_methods():`

```
  ClassDB::bind_method(D_METHOD("get_mnchar_scene"),
&Main::get_mnchar_scene);
ClassDB::bind_method(D_METHOD("set_mnchar_scene", "mnchar_scene"),
                  &Main::set_mnchar_scene);

ADD_PROPERTY(PropertyInfo(Variant::OBJECT, "packed_scene",
                        PROPERTY_HINT_RESOURCE_TYPE, "PackedScene"),
           "set_mnchar_scene", "get_mnchar_scene");
```

(Reference 8)

3. In addition, add the following code below `Main::~Main() {}`:

```
Ref<PackedScene> Main::get_mnchar_scene() { return mnchar_scene; }

void Main::set_mnchar_scene(Ref<PackedScene> packed_scene) {
mnchar_scene = packed_scene;
}
```

(Reference 8)

We're defining a scene (`mnchar_scene`) that we'll be able to access within main.cpp. Adding it as a property (like we did with movement_speed) will allow us to specify, within the editor, the exact scene (in this case, mnchar.tscn) that we want Godot to interpret as `mnchar_scene`.

4. Next, add the following to the bottom of `Main::_ready()`:

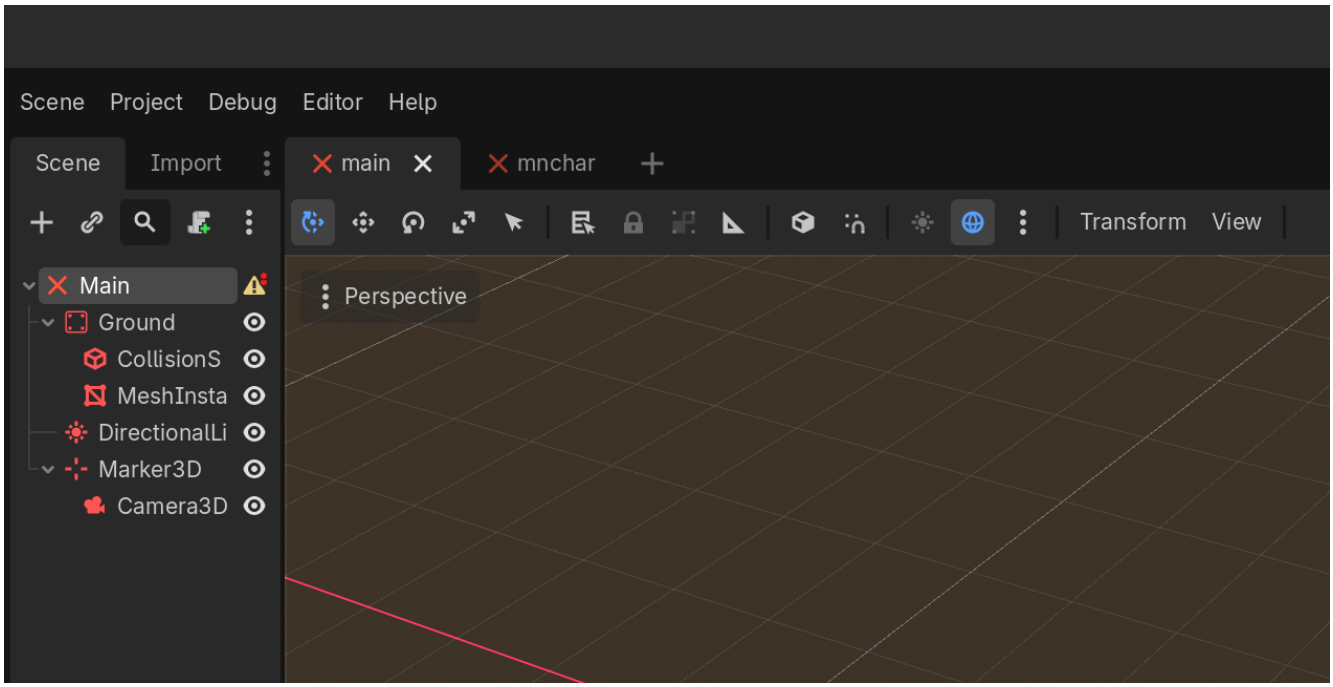
```
auto new_mnchar =
    reinterpret_cast<Mnchar *>(get_mnchar_scene()->instantiate());

add_child(new_mnchar);
```

(Reference 8)

This code retrieves our mnchar_scene; reinterprets it as a Mnchar object; and then adds it to our scene tree.

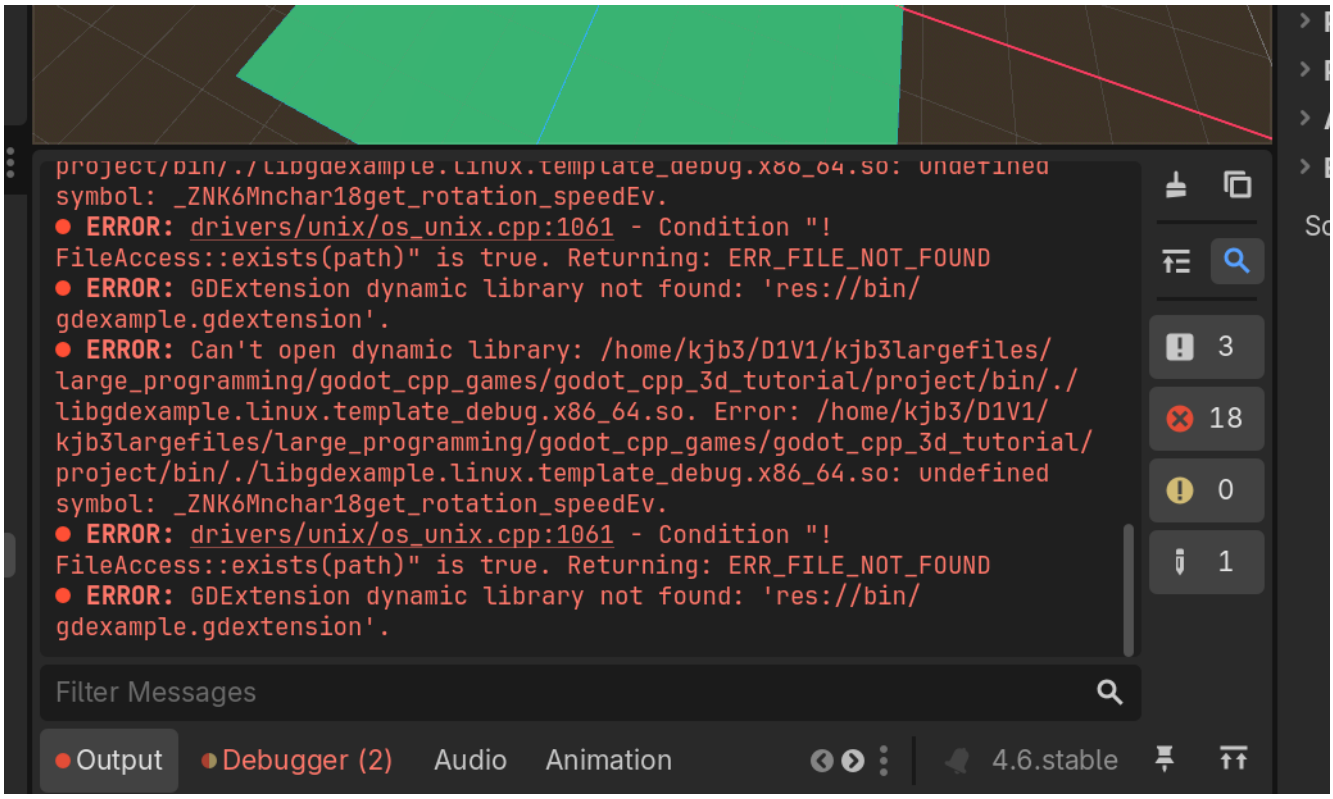
5. Note: When I was putting this code together, I initially forgot to define `get_mnchar_scene()` and `set_mnchar_scene()` within main.cpp. Although my code compiled successfully, these omissions caused Godot to fail to locate both my Main and Mnchar classes within the editor:



(I knew this was the cause because I've made similar mistakes more often than I'd like to admit!)

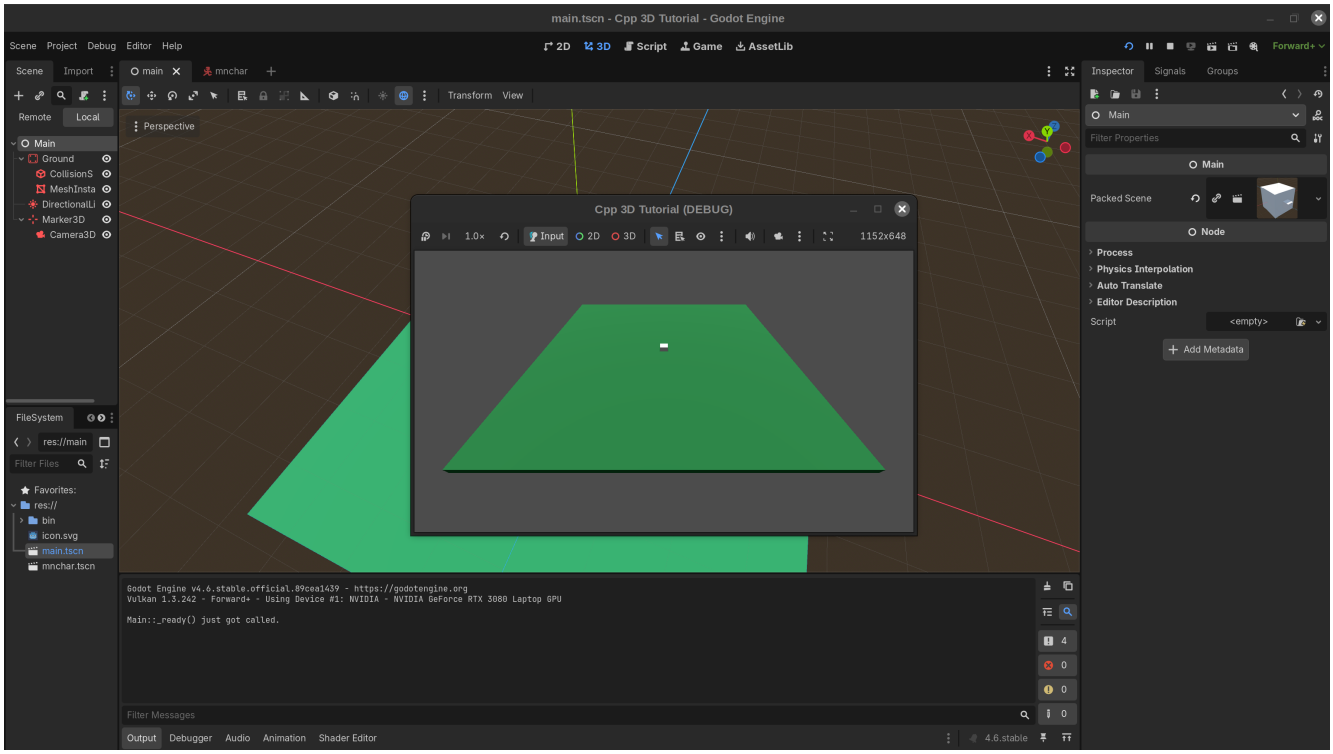
Adding these functions in, then closing and relaunching the editor resolved this issue.

I actually managed to make a similar mistake later on for `get_rotation_speed()` and `set_rotation_speed()`. Error messages within the console helped me figure out the issue. (Note the `get_rotation_speed` reference within the third-to-last error.)



6. Go ahead and compile your code, then relaunch the editor. After you open main.tscn and click on the Main node within your scene tree, you should now see a new Packed Scene entry right below Main in the inspector. Click the 'empty' text; select 'Load'; and then choose your mnchar.tscn scene.

7. When you try playing your project, you should now see a little gray box (your Mnchar) in the middle of your game area:



8. You'll notice, though, that the main character is partially sunk in the ground. We could fix this by changing its transform within Mnchar.tscn; however, because we'll need to be able to move Mnchars around later on when setting up multiplayer games, we may as well add some initial movement code now.

   Declare a new `start()` function for the Mnchar class by adding the following code right before the final closing bracket of mnchar.h:

   ```
   void start(Vector3 mnchar_translate_arg);
   ```

   Next, add the following code to the bottom of mnchar.cpp:

   ```
   void Mnchar::start(Vector3 mnchar_translate_arg)
   {translate(mnchar_translate_arg);}
   ```

   This function will ultimately allow us to configure each player's starting color, location, rotation, and ID. For now, though, we'll just use it to move our player to a better starting location.

   (Resource 8)

9. Within Main::_ready(), add the following code right above `add_child(new_mnchar)`:

   ```
   new_mnchar -> start(Vector3(15, 1, -20));
   ```

   This will move the Mnchar 15 meters to the left, one meter up (to make its bottom level with the ground), and 20 meters closer to the camera. We'll add in code later to give other Mnchars different starting locations, but all of them will also get moved up one meter.

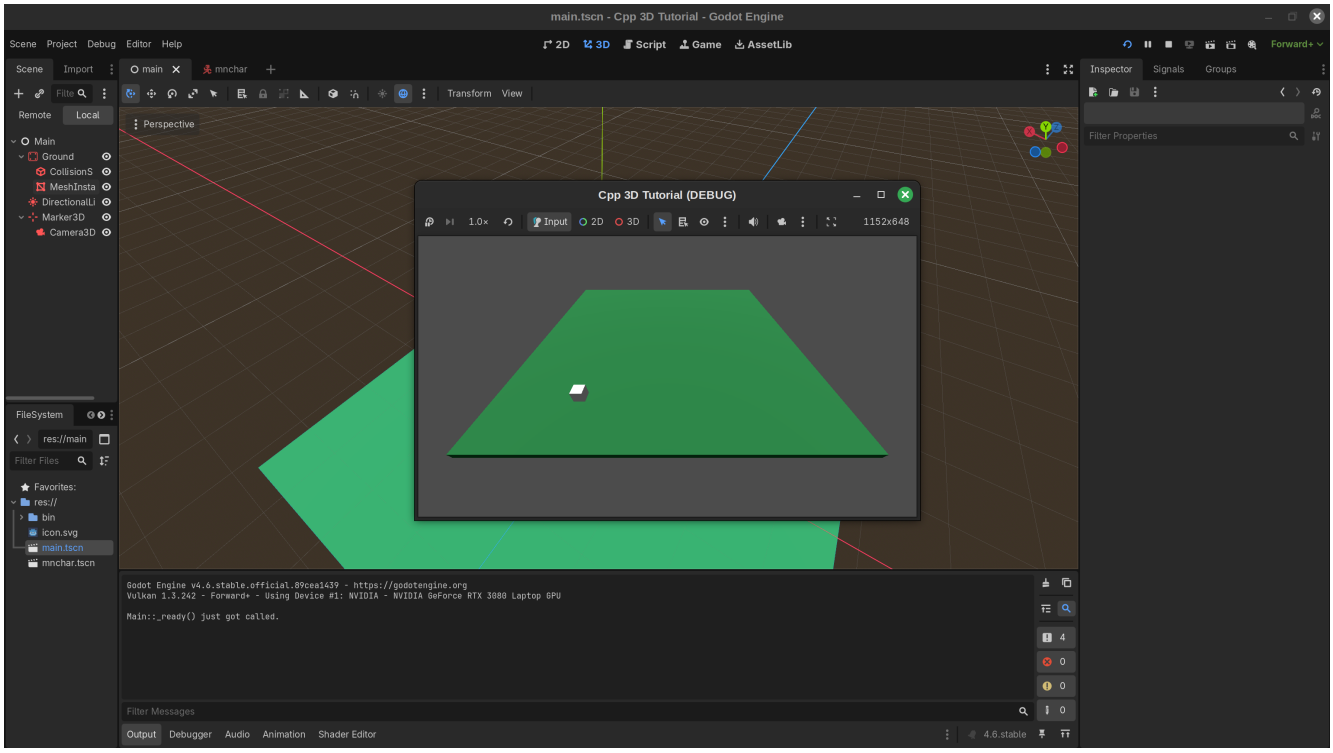
   (Resource 8)

   Note: I had originally tried this code:

   `get_node<Node3D>("Pivot") -> translate(translate_val);`

   However, this caused issues when multiple characters were added to the scene--possibly because the transforms of the actual Mnchar class weren't getting changed and were thus overlapping with one another. (This caused them to shoot up in the sky, which was both frustrating and hilarious!)

10. Compile your code, then rerun your main scene. You should now see the full Mnchar closer to the bottom left of the window:



Part 7: Moving the Mnchar

Because we're creating a multiplayer game, we'll want to set up our movement code in a way that allows each player to move his or her own Mnchar (and no one else's). The approach we'll take for this task will be as follows:

- We'll first create a new String variable (`mnchar_id`) that will store a unique ID for each Mnchar. (These IDs will range from "0" to "7" in order to support up to eight different players.) This variable will also be useful for setting character-specific locations, rotations, and colors.
- We'll also add each of these same IDs to the end of all action names within one particular group of movement commands that we'll create. Thus, one group's action names will end in "0", others will end in "1", and so forth. (This is why we added "_0" to our first set of movement names.) We'll also specify that each set of commands will only work for one particular device. (The device IDs recognized by Godot range from 0 through 7, thus matching our own list of possible IDs.)
- In our movement code, we'll check for movements that match the given Mnchar's ID. For instance, here's what our left/right movement code will look like:

```
x_direction = input->get_axis("move_left_"+mnchar_id,
"move_right_"+mnchar_id);
```

- In summary, by having Mnchar IDs match movement-name suffixes, and by having all movements for a particular suffix match only one particular device, we can create a functioning multiplayer control setup without too much extra work. (This approach was based on references 11 through 15.)

1. The first step here will be to add a `mnchar_id` value to our Mnchar class. Under `double movement_speed = 14;` within the `private` section of Mnchar.h, add:

```
String mnchar_id = "";
```

Next, in the `public` section, add the following setter and getter function declarations under your `get_movement_speed()` function declaration:

```
void set_mnchar_id(const String mnchar_id);
String get_mnchar_id() const;
```

Finally, add a `mnchar_id_arg` argument before your `mnchar_translate_arg` within `start()` so that the declaration matches the following line:

```
void start(String mnchar_id_arg, Vector3 mnchar_translate_arg);
```

2. Within mnchar.cpp, add the following below your `get_movement_speed()` function definition:

```
void Mnchar::set_mnchar_id(const String p_mnchar_id) {
mnchar_id = p_mnchar_id;
}

String Mnchar::get_mnchar_id() const { return mnchar_id;}
```

Next, add `String mnchar_id_arg` before `Vector3 mnchar_translate_arg` within the arguments in this file's `Mnchar::start()` function definition. In addition, right before `translate(mnchar_translate_arg);` in the function body, add: `set_mnchar_id(mnchar_id_arg);`.

3. Finally, within main.cpp, update your `new_mnchar -> start()` command within `Main::_ready()` so that it reads as follows:

```
new_mnchar -> start("0", Vector3(15, 1, -20));
```

(This will assign `new_mnchar` an ID of 0, thus allowing all movement commands ending in "0" to get registered by this Mnchar.)

If you want to confirm that this change has taken effect, you can also add the following code following `set_mnchar_id(mnchar_id_arg)` within `Mnchar::start()`:

```
UtilityFunctions::print("Mnchar's ID is ", get_mnchar_id(), ".");
```

(Printing out values is incredibly helpful for debugging work.)

We could also have added `bind_method()` and `ADD_PROPERTY()` calls for our `mnchar_id` value and its corresponding setter and getter functions (as we did with the `movement_speed` variable). However, that won't be necessary in this case, as we won't need to access or modify those values directly within the editor.

4. Now that we have code in place for assigning `mnchar_id`s, we can utilize that ID within our input code. First, add the following code after your `start()` function declaration within mnchar.h:

```
void _physics_process(double delta) override;
```

(See Reference 4 for the use of `_physics_process` here rather than `_process`.)

In addition, add the following three include statements after `#include <godot_cpp/variant/utility_functions.hpp>`:

```
#include <godot_cpp/classes/input.hpp>
#include <godot_cpp/classes/input_event.hpp>
#include <godot_cpp/classes/input_map.hpp>
```

5. Next, add the following code to the end of mnchar.cpp. We'll start each `_physics_process` call by retrieving input data that we can then parse. (The lack of a closing bracket is intentional, since we'll be filling out the rest of this function below.)

```
void Mnchar::_physics_process(double delta) {

auto input = Input::get_singleton();
```

(Reference 4)

6. Extend this function by adding the following code, which uses our left, right, forward, and back movements to determine the player's movement along the x (left/right) and z (forward/back) axes. This

approach will allow different movement speeds depending on exactly how far a controller joystick is moved down, but it also works fine for keyboard input.

(Note: I've found that I sometimes need to put 'move_right' before 'move_left', and 'move_forward' before 'move_back', in order to get my movement code to work correctly.)

```
float x_direction =
    input->get_axis("move_left_" + mnchar_id, "move_right_" + mnchar_id);
float z_direction =
    input->get_axis("move_forward_" + mnchar_id, "move_back_" +
mnchar_id);
```

Also note the inclusion of mnchar_id, which I discussed at length earlier.

(References 16 and 17)

7. Next, we'll rotate the player in response to any rotation commands sent by the player's controller (or, if `mnchar_id` is 0, the keyboard). Add the following code to the end of the function:

```
get_node<Node3D>("Pivot")->rotate_object_local(
    Vector3(0, 1, 0),
    rotation_speed * input->get_axis("rotate_right_" + mnchar_id,
                                     "rotate_left_" + mnchar_id));
```

(References 16, 18, and 19)

8. We'll now retrieve information about the Mnchar's basis that we'll use to update its velocity. Add the following code to the end of `_physics_process()`:

Note: I'm not sure why, but multiplying the x and z components of the x and z bases, respectively, by -1 was critical for getting the movement code to work. This may be due to an issue with my setup. I imagine that there's a way to update my code such that at least one of these multiplication commands won't be necessary.

```
auto player_transform_basis_z =
    get_node<Node3D>("Pivot")->get_transform().get_basis()[2];

auto player_transform_basis_x =
    get_node<Node3D>("Pivot")->get_transform().get_basis()[0];

player_transform_basis_x.x *= -1;

player_transform_basis_z.z *= -1;
```

9. Extend the function by adding the following code, which initializes, then updates, the player's target velocity.

```
Vector3 target_velocity = Vector3(0, 0, 0);

float abs_z_direction = std::abs(z_direction);
float abs_x_direction = std::abs(x_direction);

if (abs_z_direction >= abs_x_direction) {
    target_velocity +=
        -1 * player_transform_basis_z * z_direction * movement_speed;
} else {
    target_velocity +=
        -1 * player_transform_basis_x * x_direction * movement_speed;
}
```

   I would like Mnchar to be able to move *only* forward, back, left, or right relative to its current position (i.e. not diagonally). Therefore, the code above finds the absolute value of the x and z directions, then moves the player along the axis with the greatest absolute value. This isn't strictly necessary, but I find it makes the Mnchar's movement somewhat more intuitive.

   (Note that I'm using the standard library's abs() function rather than Godot's, as the latter appeared to truncate values down to the nearest int.)

   (References 4, 17, 20, and 21)

10. Finally, close out this function with the following code:

```
set_velocity(target_velocity);

move_and_slide();
}
```

    (References 4 and 21)

11. Reopen your editor, launch the scene, and test out the controls. Make sure that no directions are the inverse of what you'd expect--and that the player cannot move diagonally.

    By the way: if the game crashes right when you launch it, make sure that your mnchar.tscn scene is still present within Main's Packed Scene attribute. (It sometimes disappears on my end, but thankfully, it's easy to add back in.)

Part 8: Creating a projectile

It's neat to move our Mnchar around with code, but the game won't be too much fun without anything for it to fire (or be hit by). Therefore, let's go ahead and add a Projectile class to our game. This class will have many similarities to the Mnchar class (on which its code will be based).

1. Within your src/ folder, create two new files, 'projectile.h' and 'projectile.cpp'. Add the following text to projectile.h:

```
#pragma once

#include <godot_cpp/classes/character_body3d.hpp>
#include <godot_cpp/core/class_db.hpp>
#include <godot_cpp/variant/utility_functions.hpp>

using namespace godot;

class Projectile : public CharacterBody3D {
GDCLASS(Projectile, CharacterBody3D)

private:
double projectile_speed = 64;
double active_time = 0;
String firing_mnchar_id = "";
Vector3 projectile_velocity = Vector3(0, 0, 0);

protected:
static void _bind_methods();

public:
Projectile();
~Projectile();

void set_projectile_speed(const double movement_speed);
double get_projectile_speed() const;

void set_firing_mnchar_id(const String firing_mnchar_id_arg);
String get_firing_mnchar_id() const;

void start(Transform3D transform, String firing_mnchar_id);
void _physics_process(double delta) override;
};
```

`active_time` will store how long a projectile has been active, thus allowing us to remove it from the scene after a certain amount of time has passed. This prevents projectiles from existing in the game's memory forever, which would prove to be inefficient. (Reference 22)

`firing_mnchar_id` will let us determine which Mnchar fired a projectile--and, thus, which Mnchar should be credited for a hit on another Mnchar. We won't use this variable for a little while, but it doesn't hurt to add it in now.

2. Next, add the following code to projectile.cpp:

```
#include "projectile.h"

using namespace godot;

void Projectile::_bind_methods() {
ClassDB::bind_method(D_METHOD("get_projectile_speed"),
                     &Projectile::get_projectile_speed);
ClassDB::bind_method(D_METHOD("set_projectile_speed",
"p_projectile_speed"),
                     &Projectile::set_projectile_speed);
ADD_PROPERTY(PropertyInfo(Variant::FLOAT, "projectile_speed"),
             "set_projectile_speed", "get_projectile_speed");
}

Projectile::Projectile() {}

Projectile::~Projectile() {}

void Projectile::set_projectile_speed(const double p_projectile_speed) {
projectile_speed = p_projectile_speed;
}

double Projectile::get_projectile_speed() const { return
projectile_speed; }

void Projectile::set_firing_mnchar_id(const String firing_mnchar_id_arg)
{
firing_mnchar_id = firing_mnchar_id_arg;
}

String Projectile::get_firing_mnchar_id() const { return
firing_mnchar_id; }
```

This is all fairly similar to what we've added within mnchar.cpp.

3. Add the following code right below your existing projectile.cpp code:

```
void Projectile::start(Transform3D transform, String firing_mnchar_id) {

set_firing_mnchar_id(firing_mnchar_id);
```

```
set_transform(transform);

auto projectile_basis_z = Projectile::get_transform().get_basis()[2];

projectile_basis_z.z *= -1;

projectile_velocity = -1 * projectile_basis_z * projectile_speed;
}
```

This code will allow us to configure a new Projectile that a Mnchar has just fired. In conjunction with an upcoming update to mnchar.cpp, it will set the Projectile's `firing_mnchar_id` with that Mnchar's `mnchar_id`; set the Projectile's transform based on that Mnchar's transform; and initialize the projectile's velocity. (This velocity code is quite similar to that found within `Mnchar::_process()`. One difference is that we don't need to calculate and then incorporate x_direction and z_direction values; the z direction will always be 1, and the x direction will always be 0.

(Based on references 18, 23, 24, 27, and 28. References 25 and 26 are also helpful for projectile-related code.)

4. Finally, add the following code to the bottom of projectile.cpp:

```
void Projectile::_physics_process(double delta) {
auto collision = move_and_collide(projectile_velocity * delta);
if (active_time >= 2) {
    queue_free();
}
active_time += delta;
}
```

The queue_free() command removes this particular projectile from the game, which we'll want to do after a certain amount of time (in this case, two seconds) has elapsed. It would be ideal to make this time contingent on the size of the game area (which probably won't change very much) and the projectile's movement speed, but this simpler apporach will work OK for now.

(Based on references 8 and 23)

5. As always, before we can incorporate this class into our game, we'll need to add references for it within register_types.cpp. Within that script, add `#include "projectile.h"` below `#include "mnchar.h"`, and `GDREGISTER_RUNTIME_CLASS(Projectile);` below `GDREGISTER_RUNTIME_CLASS(Mnchar);`.

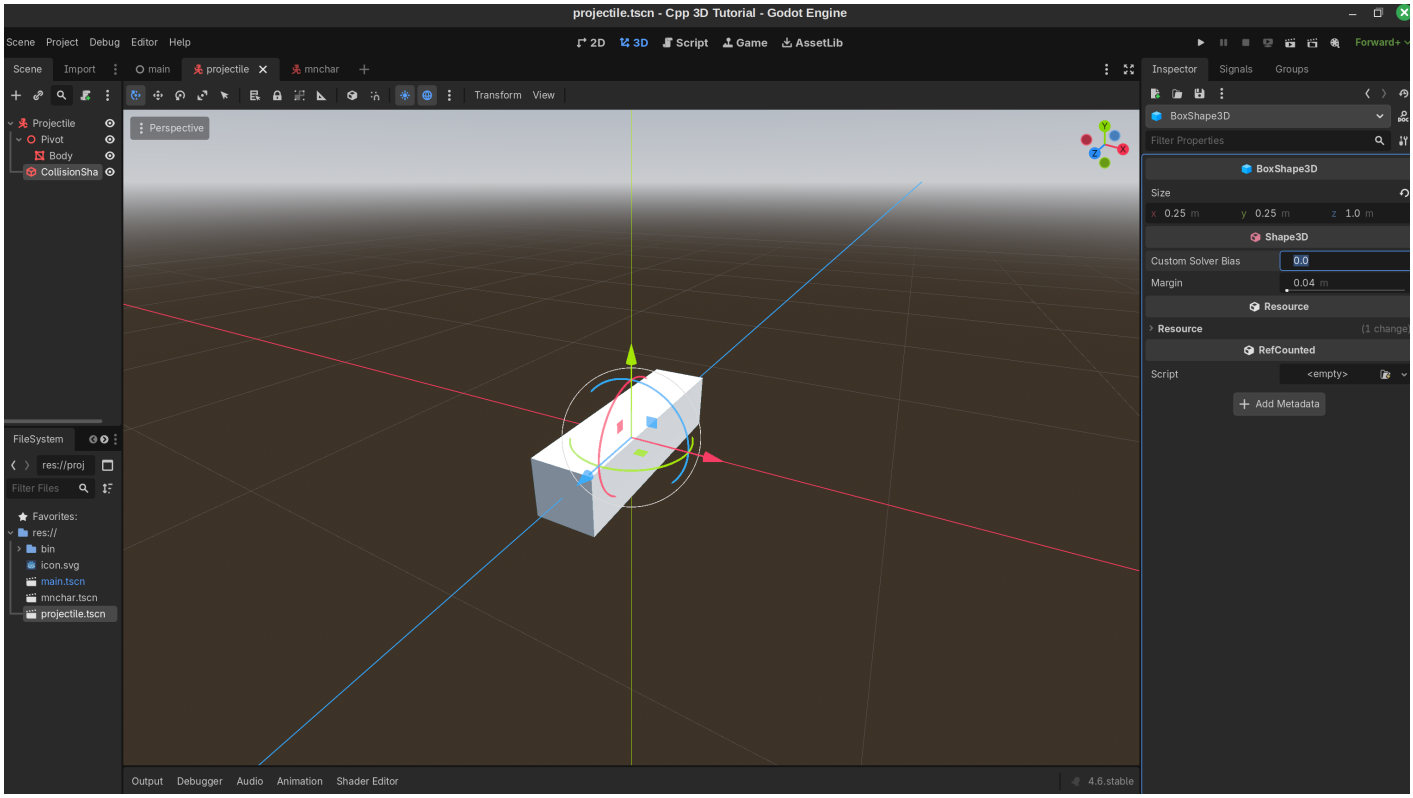
6. We'l have more to add to our Projectile class's code later on, but what we have so far will at least let us create a Projectile scene within our editor. Go ahead and compile your code, then restart your editor.

Part 9: Adding a projectile scene

1. Select Scene --> New Scene; click 'Other Node' under the 'Create Root Node:' text; and search for your new Projectile class. Once you've found it, select it and hit 'Create.' Go ahead and save this near-empty scene as projectile.tscn.

2. Just as our Projectile code was based on our Mnchar code, our Projectile scene will have many similarities to our Mnchar scene. As you did within mnchar.tscn, add a Node3D as a child of your Projectile and rename it 'Pivot.' Then add a MeshInstance3D as a child of this Pivot; rename it 'Body'; and assign it a BoxMesh within the MeshInstance3D section of the Inspector. Within the edit menu for this BoxMesh, create a new StandardMaterial3D for it, then set both its x and y Size values equal to 0.25. (Leave the z value unchanged at 1.) (Refer to the 'Adding a Mnchar to the game area' section of the tutorial if you've forgotten how to perform any of these steps.)

3. Finally, add a CollisionShape3D as a child of Projectile; assign it a BoxShape3D; and set this shape's x, y, and z values to 0.25, 0.25, and 1, respectively. (Again, refer to the 'Adding a Mnchar to the game area' section if needed.)



Part 10: Allowing Mnchars to fire projectiles

Next, we'll need to add code for firing projectiles to our Mnchar class.

1. Within mnchar.h, add the following two lines to the end of your list of `#include` statements:

```
#include <godot_cpp/classes/packed_scene.hpp>
#include "projectile.h"
```

2. Next, add `void shoot_projectile();` right above `void _physics_process(double delta) override;`. Then add `Ref<PackedScene> projectile_scene;` at the end of your `private` section (right after `String mnchar_id = "";`). Finally, add the following code rigth before `shoot_projectile()` within the `public` section of this file:

```
Ref<PackedScene> get_projectile_scene();
void set_projectile_scene(Ref<PackedScene>);
```

This code will ultimately allow us to access the Projectile scene within our Mnchar class. It's very similar to the code we're using to access Mnchars themselves within the Main class.

3. Within mnchar.cpp, add the following code to the end of `Mnchar::_bind_methods()` so that we can access this `projectile_scene` variable within the editor (and thus link projectile.tscn to it):

```
ClassDB::bind_method(D_METHOD("get_projectile_scene"),
                     &Mnchar::get_projectile_scene);
ClassDB::bind_method(D_METHOD("set_projectile_scene",
"projectile_scene"),
                     &Mnchar::set_projectile_scene);
ADD_PROPERTY(PropertyInfo(Variant::OBJECT, "packed_scene",
                              PROPERTY_HINT_RESOURCE_TYPE, "PackedScene"),
             "set_projectile_scene", "get_projectile_scene");
```

4. Next, right below `Mnchar::~Mnchar() {}`, define `projectile_scene`'s setter and getter functions as follows:

```
Ref<PackedScene> Mnchar::get_projectile_scene() { return
projectile_scene; }

void Mnchar::set_projectile_scene(Ref<PackedScene> packed_scene) {
projectile_scene = packed_scene;
}
```

(Based on reference 8)

5. Next, within mnchar.cpp, add the following definition of this function right after your `Mnchar::start()` function:

```
{
auto projectile =
```

```
    reinterpret_cast<Projectile *>(projectile_scene->instantiate());

Transform3D projectile_transform =
    get_node<Node3D>("Pivot")->get_global_transform();

projectile_transform =
    projectile_transform.translated_local(Vector3(0, 0, 3));
projectile->start(projectile_transform, mnchar_id);
get_parent()->add_child(projectile);
}
```

We need to initialize the projectile's transform as that of the main character's Pivot node because it's this node, not Mnchar itself, whose basis we adjust when moving the player.

The `translated_local()` call creates some distance in between the projectile and the firer. This prevents the firer from getting hit by its own bullet immediately after firing! (We're using `translated_local()` rather than `translate()` here because we want to move the projectile in front of the Mnchar's field of view rather than the game area itself.)

Adding `get_parent()` before `add_child()` ensures that these projectiles are children of the parent scene (e.g. main.tscn) rather than the character. Without this line, projectiles might rotate whenever the character rotates--which, while a potentially-cool mechanic, isn't what we're looking for here.

(Based on references 8, 18, 24, 29, 30, and 31)

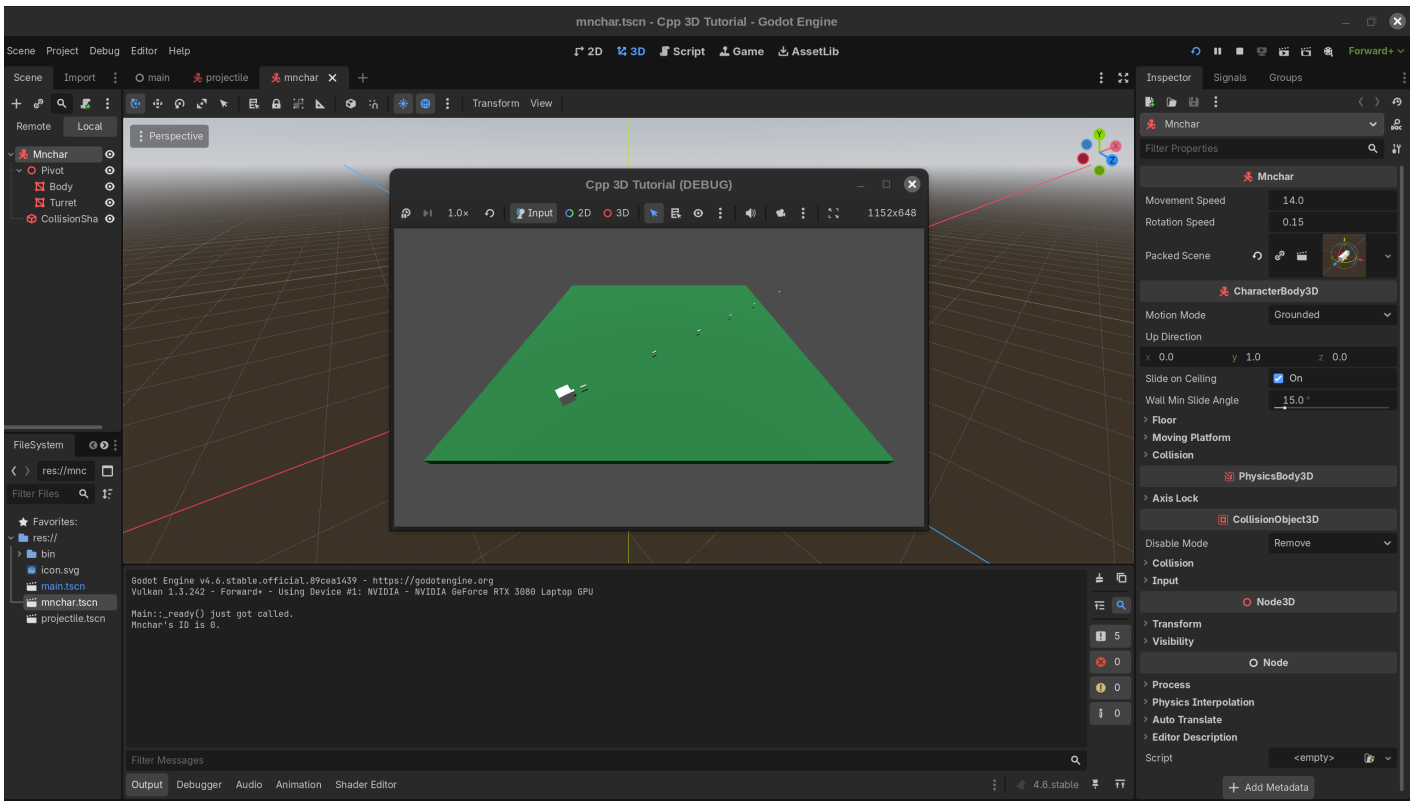
6. Now that we have a function for firing projectiles, we also need to allow the player to trigger (pun intended?) it. We can do so by adding the following code right after `auto input = Input::get_singleton();` within `Mnchar::_physics_process`:

```
if ((input->is_action_just_pressed("fire_" + mnchar_id)) &&
    (input->is_action_pressed("reset_" + mnchar_id) == false)) {
    shoot_projectile();
}
```

Later in this tutorial, we'll allow players to start a new game by pressing both their 'fire' and 'reset' buttons. Therefore, I added code to this `if` statement that will only allow a projectile to get fired when reset is *not* being pressed. (Otherwise,the game would always start with a projectile being fired, which could give the firing player an advantage and/or distort any accuracy statistics that we might choose to collect.)

7. Compile your code, then restart your editor. When you click on mnchar.tscn, you should see a new 'Packed Scene' attribute below 'Movement Speed' and 'Rotation Speed' near the top of the editor. (It's neat how these variables get formatted automatically to look nicer within the editor, by the way.) Click the folder icon to the right of 'empty', then select projectile.tscn.

8. Launch your game. Confirm that pressing space bar causes a projectile to fire in front of the player's turret. Also confirm that this remains the case regardless of which direction the player is facing, and that the projectile's velocity isn't affected in any way by the player's actions following the fire command. (It took quite a bit of debugging on my part when I was initially coding this section to get things working. I hope that you won't need to do the same, but don't get too frustrated if things don't work right away.)



Part 11: Adding a second Mnchar to the scene

Now that we've added code for firing a projectile, we'll also need to create code that specifies how the game should react when a Mnchar gets hit by a projectile. But in order to test out that code, we'll need to add a second Mnchar to the scene.

Since having two identical Mnchars within a scene can get confusing, this will also be a good time to create code that assigns different colors to different players. And in order to assign those colors, we may as well set up a typed dictionary that will store colors for all the players who will might eventually join our game.

Finally, since we're adding a typed dictionary for player colors, we may as well add ones for starting locations and rotation values as well, as these will also play an important role in initializing new Mnchars.

(This is a bit like If You Give a Mouse a Cookie, but in game-development form! It's funny quickly one programming task can lead to five others.)

1. With that rambling introduction out of the way, let's go ahead and create typed dictionaries that will store colors, rotation values, and starting locations for all of our players. Within main.h, add `#include <godot_cpp/variant/typed_dictionary.hpp>` to the bottom of your list of `#include`

`statements`. Next, enter the following code right below `Ref<PackedScene> mnchar_scene` within the `private` section:

```
TypedDictionary<String, Vector3> mnchar_id_location_dict{
    {String("0"), Vector3(15, 0, -20)},  {String("1"), Vector3(-15, 0,
20)},
    {String("2"), Vector3(-20, 0, -15)}, {String("3"), Vector3(20, 0,
15)},
    {String("4"), Vector3(-5, 0, -20)},  {String("5"), Vector3(5, 0,
20)},
    {String("6"), Vector3(-20, 0, 5)},   {String("7"), Vector3(20, 0,
-5)}};

TypedDictionary<String, double> mnchar_id_rotation_dict{
    {String("0"), 0},           {String("1"), Math_PI},
    {String("2"), Math_PI / 2}, {String("3"), Math_PI / -2},
    {String("4"), 0},           {String("5"), Math_PI},
    {String("6"), Math_PI / 2}, {String("7"), Math_PI / -2}};

TypedDictionary<String, Color> mnchar_id_color_dict{
    {String("0"), Color(0, 0, 1, 1)}, {String("1"), Color(0, 1, 0, 1)},
    {String("2"), Color(0, 1, 1, 1)}, {String("3"), Color(1, 0, 0, 1)},
    {String("4"), Color(1, 0, 1, 1)}, {String("5"), Color(1, 1, 0, 1)},
    {String("6"), Color(1, 1, 1, 1)}, {String("7"), Color(0, 0, 0, 1)}};
```

This code is based on references 32 (an overview of Godot's own core types) and 33 (the example.cpp file within the godot-cpp repository). Both of these resources proved very helpful in working with Godot types like TypedDictionary, so I highly recommend checking them out if you aren't already familiar with them. The syntax is also similar to that for `std::map` (see reference 35).

Each of these dictionaries uses string-based IDs as keys and various initialization settings (namely, starting locations, starting rotations, and colors) as values. The transforms and rotations allow for an adequate amount of space between players while keeping them all facing towards the game area. (I also needed to make sure that no player would begin in another's direct line of fire.)

The rotation values are in radians; they use the MATH_PI variable found within reference 34. For the rotation dictionary, the fourth value within each 'Color' represents that color's opacity.

2. Next, we need to update Mnchar::start() in order to utilize these rotation and color variables. Update your `void start()` function within mnchar.h so that it reads:

```
void start(String mnchar_id_arg, Vector3 mnchar_translate_arg,
         double mnchar_rotation_arg, Color mnchar_color_arg);
```

3. We'll also need to declare and define a function that can modify a Mnchar's color. Right above `void shoot_projectile();` within the `public` section of mnchar.h, add:

```
void set_mnchar_color(const Color mnchar_color_arg);
```

4. Switch over to mnchar.cpp. Add the following two statements to the end of your `#include` list:

```
#include <godot_cpp/classes/base_material3d.hpp>
#include <godot_cpp/classes/mesh_instance3d.hpp>
```

5. Then, after your `Mnchar::get_mnchar_id()` definition within mnchar.cpp, define this function as follows:

```
void Mnchar::set_mnchar_color(const Color mnchar_color_arg) {
UtilityFunctions::print("Mnchar::set_mnchar_color() checkpoint 1.");
Ref<BaseMaterial3D> mncharbody_mesh_material_3d = (
    get_node<Node3D>("Pivot")
        ->get_node<MeshInstance3D>("Body")
        ->get_mesh()
        ->surface_get_material(0));

mncharbody_mesh_material_3d->set_albedo(mnchar_color_arg);

get_node<Node3D>("Pivot")
    ->get_node<MeshInstance3D>("Body")
    ->get_mesh()
    ->surface_set_material(0, Ref<Material>
(mncharbody_mesh_material_3d));
    }
```

This code (Based on references 36, 37, 38, and 39--special thanks to RamblingStranger and pescador in the Godot discord for their help here) first retrieves the current material of the Mnchar's Pivot. It then uses `Ref<>` to create a `BaseMaterial3D` copy of this material, as this class has a `set_albedo` method that we can use to convert the existing material to our new color. Finally, it sets the pivot's existing material to a `Material` copy of this BaseMaterial3D.

(pescador noted in Discord that "Ref wrappers automatically verify and give you the right type to you", but also clarified that this approach "only works for RefCounted objects." For an alternative approach that may be more flexible, see the `Mnchar::set_character_color()` method within mnchar.cpp in Reference 2.)

Also note that, for this code to work, the "Pivot" and "Body" names *must* also be present within your scene tree within Mnchar.tscn. If you're using a different name for one of these items, your game will most likely crash before either Mnchar appears (which I know from experience!).

6. Replace the `Mnchar::start()` function definition within mnchar.cpp with the following code:

```
void Mnchar::start(String mnchar_id_arg, Vector3 mnchar_translate_arg,
                double mnchar_rotation_arg, Color mnchar_color_arg)
{
set_mnchar_id(mnchar_id_arg);
UtilityFunctions::print("Mnchar's ID is ", get_mnchar_id(), ".");
translate(mnchar_translate_arg);
set_mnchar_color(mnchar_color_arg);
get_node<Node3D>("Pivot")->rotate_object_local(Vector3(0, 1, 0),
                                                mnchar_rotation_arg);
}
```

This function will now not only set the Mnchar's ID and starting location, but also update its color and rotation.

7. Next, replace your existing Main::_ready() {} function definition with the following text:

```
void Main::_ready() {

    UtilityFunctions::print("Main::_ready() just got called.");

    Array mnchars_to_include {"0", "1"};
    for (int index = 0; index < mnchars_to_include.size(); index++)
    {
    String mnchar_id_arg = mnchars_to_include[index];
    Color mnchar_color_arg = mnchar_id_color_dict[mnchar_id_arg];
    Vector3 mnchar_translate_arg =
mnchar_id_location_dict[mnchar_id_arg];
    double mnchar_rotation_arg = mnchar_id_rotation_dict[mnchar_id_arg];

    auto new_mnchar = reinterpret_cast<Mnchar *>(
        get_mnchar_scene()->instantiate());

    new_mnchar -> start(mnchar_id_arg, mnchar_translate_arg,
        mnchar_rotation_arg, mnchar_color_arg);

    add_child(new_mnchar);
    }

}
```

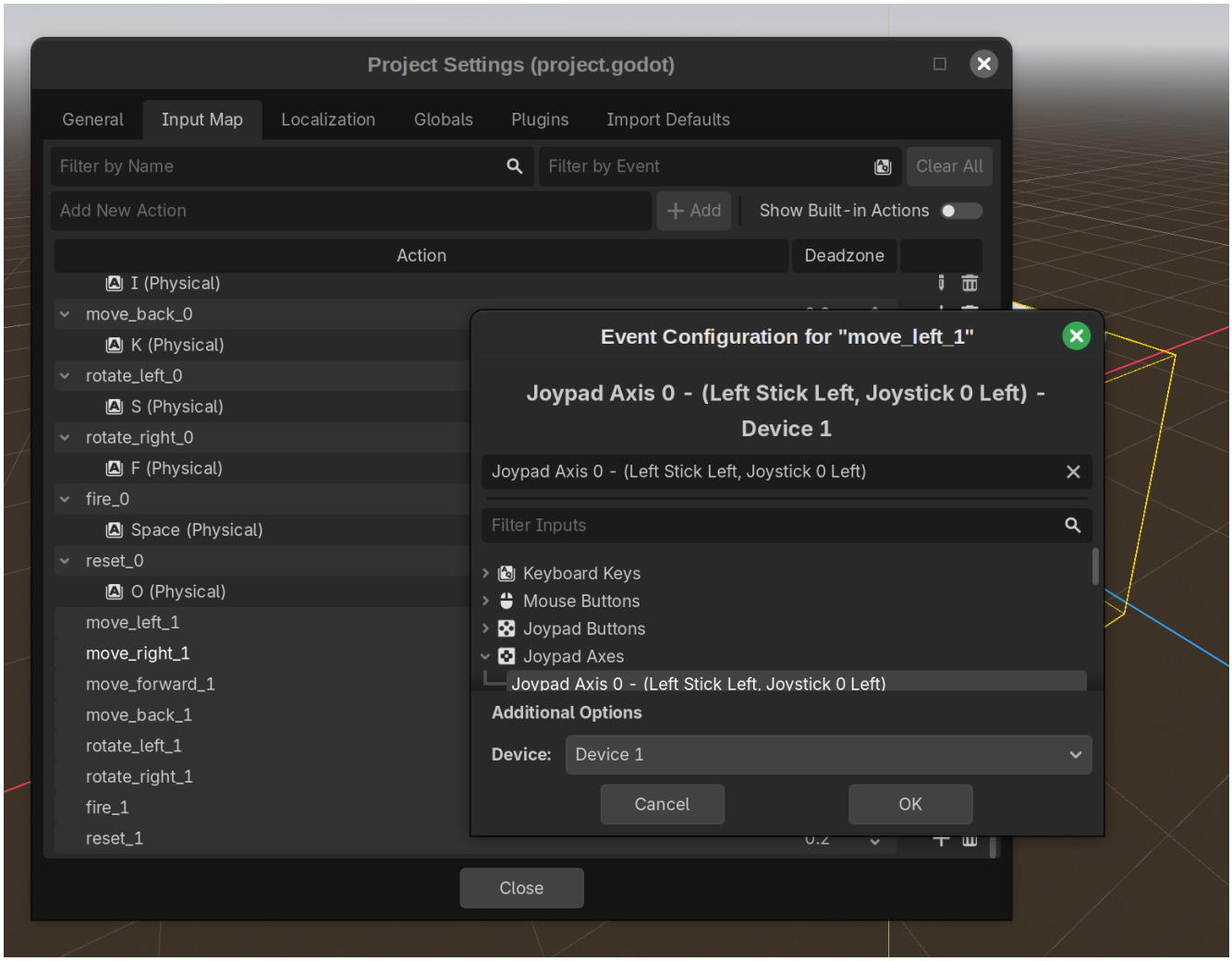
We're now adding new Mnchars to the game via a loop that iterates through an array of Mnchar IDs. These IDs are used to specify each player's color, translation, and rotation, all of which (together with the ID) then get passed to our `Mnchar::start()` function.

The `mnchars_to_include` Array will eventually get passed to this function, but for now, we'll store hardcoded "0" and "1" values within it..

8. Go ahead and compile your code, then restart the editor. Now that we have a second Mnchar within our game, we should add in inputs for this player. Go to Project --> Project Settings --> Input Map to access the input editor.

9. If you happen to have a game controller with two joysticks and left/right triggers available, connect it to your computer now. (If you don't, no worries--I'll show you another way to add these controls.) For the move_left_1, move_right_1, move_forward_1, and move_back_1 actions, move your left joystick left, right, forward, and back to add them as inputs. For the rotate_left and rotate_right actions, move your right joystick to the left and to the right. Finally, for the fire_1 and reset_1 commands, press the right and left triggers, respectively.

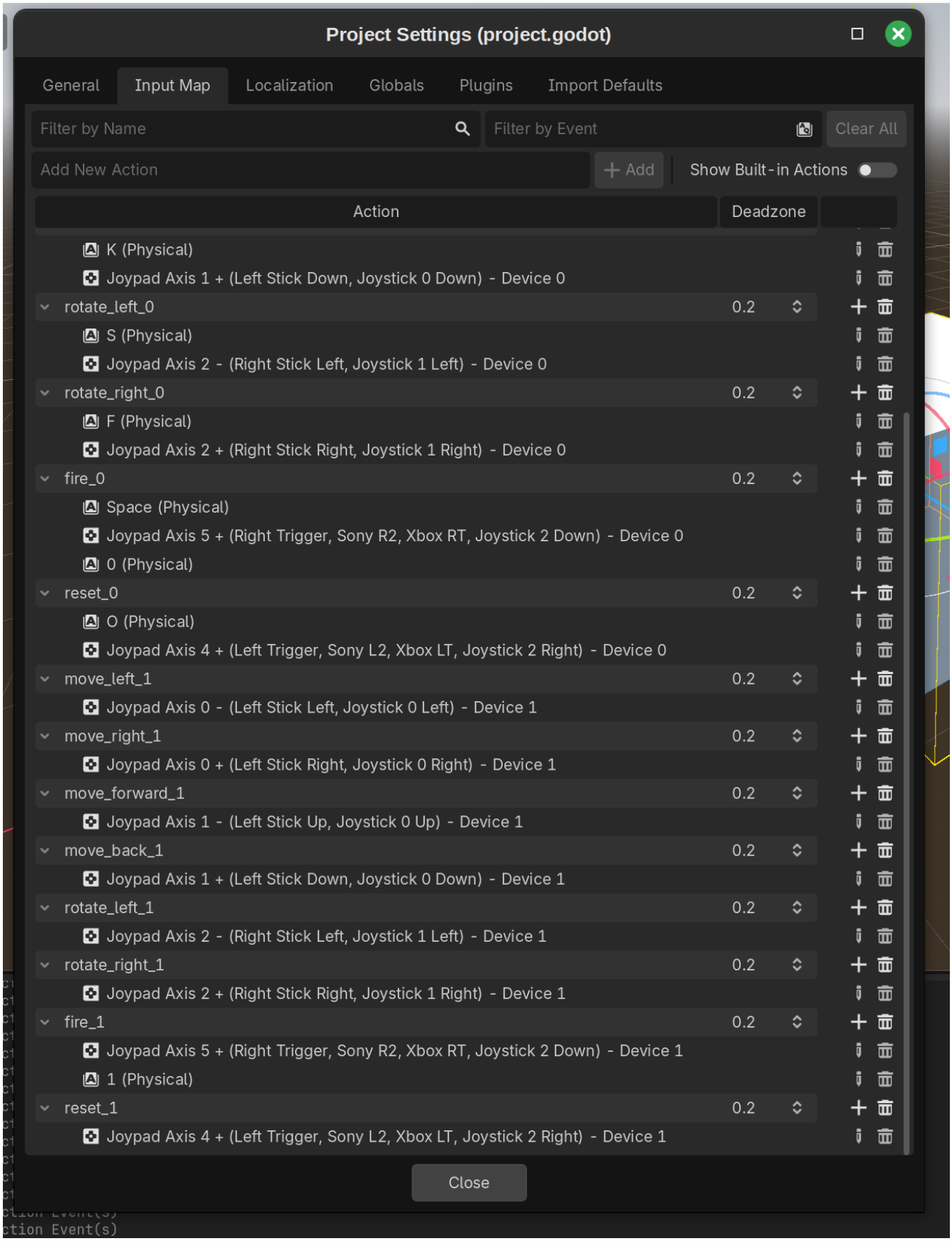
   When adding in each of these actions, make sure to specify Device 1 (not 'All devices') as the input device. Otherwise, these actions will also affect the movement of Mnchar 0.



You can also peform similar steps for your existing actions that end in '_0'. Just make sure to select Device 0 for those rather than Device 1.

In order to make debugging easier, link the '0' key on your keyboard to the 'fire_0' command, and the '1' key to 'fire_1'. (You'll see why I recommend this soon.)

Once you've made these changes, the lower section of your input map should look like the following:

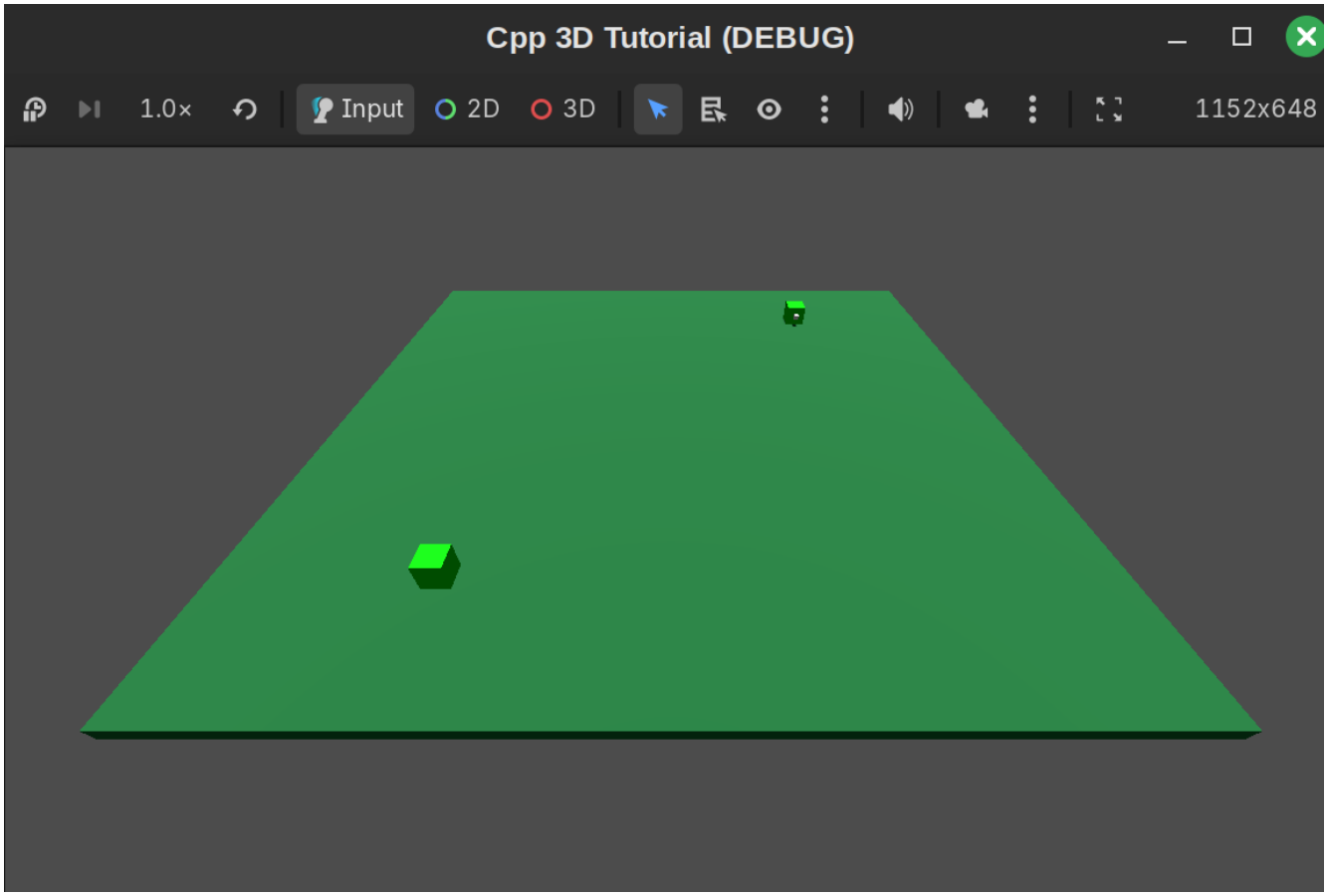


If you don't have a controller, or simply don't want to go through the trouble of adding in these commands, you can *also* go to this repository's project.godot file (https://github.com/kburchfiel/godot_cpp_3d_tutorial/blob/main/project/project.godot); copy all of the

text between the `[input]` entry and the [`physics`] entry; and then paste it into your own project's project.godot file. (This file should be available at /project/project.godot.)

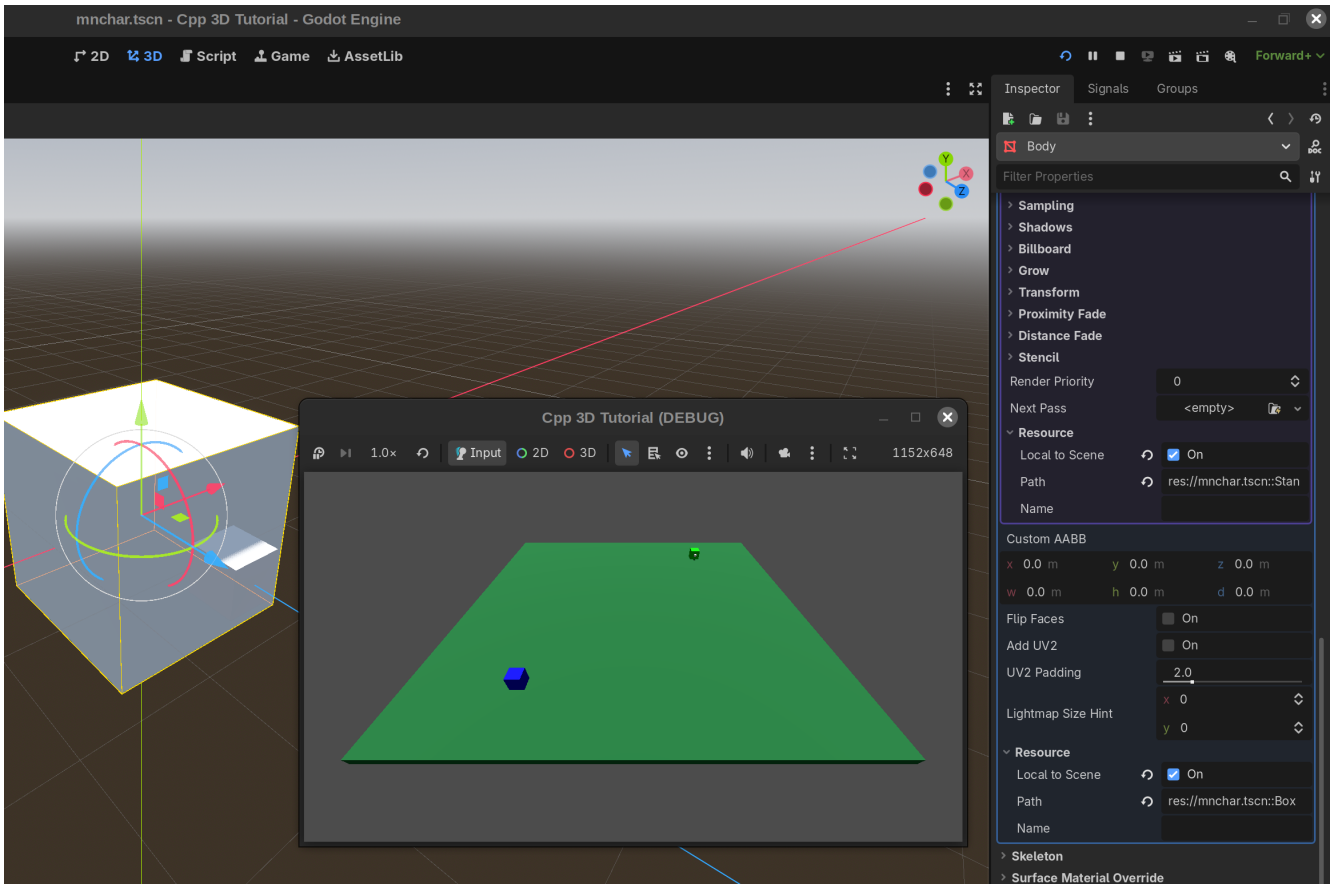
10. If you just really, *really* love adding these inputs, you can enter similar commands for device IDs 2 through 7 manually. However, I highly recommend that you instead copy those actions from the previous link and paste them into your project.godot file. (By the way, I created most of these inputs, aside from the keyboard-based ones, via another set of C++ code. See Reference 47 for the link.) Once you've finished these copy/paste commands, make sure that they're appearing within your input map.

11. Launch your main scene. You should see the following:



Two players are now present within the game area, as expected. But why are they both green? Was there an issue with our color dictionary? After quite a bit of debugging, I eventually found the solution in a post by Tobias Wink (Reference 40). The problem is that, currently, our Mnchar's material is shared across both of our Mnchar instances; as a result, changing the color of the second Mnchar will also change the first Mnchar's color.

To resolve this, double-click on your mnchar.tscn scene; click on the 'Body' MeshInstance3D; click the white cube within the Mesh section of the Inspector (not the game) to open the Edit window; open up the Resource section near the bottom; and check the 'Local to Scene' box. Next, click the white sphere within the Material section of the editor; scroll down to and open its own Resource section; and check *that* 'Local to Scene' box as well. After saving your scene and relaunching your game, you should now see one blue cube and one green cube within the game area:



12. As the blue Mnchar, try firing some projectiles towards the green Mnchar by pressing the space bar. (If the game crashes when you attempt to do so, make sure that your projectile.tscn is still showing up within the Mnchar's Packed Scene entry; if it's missing, load it back in.) You should see the projectiles stop in place when they reach the green Mnchar, though if enough of them get fired, it might shift it around a bit. We'll now change this behavior so that a Mnchar who gets hit by a projectile gets removed from the game scene.

Part 12: Detecting collisions

1. In order to determine when a Mnchar has gotten hit, we'll need to add an Area3D node that can detect such collisions. Go ahead and add such a node as a child of your Mnchar, then rename it 'Projectile_Detector.' Next, add a CollisionShape3D as a child of Projectile_Detector; click on the 'empty' text next to the Shape section within its Inspector menu; and choose a BoxShape3D. Click on the 'BoxShape3D' text to enter the edit menu, then set the x, y, and z sizes to 2.0 meters. (Reference 41)

2. Go back to your Projectile_Detector node and deselect the 'Monitoring' property within the Inspector (but keep 'Monitorable' active). Next, click 'Collision' within the CollisionObject3D section of the Projectile_Detector's Inspector menu in order to open up its Layer and Mask settings. Deselect the '1' within the Layer and Mask sections, then select the '2'. This will allow the detector to recognize collisions only with objects with a Layer value of 2. (Reference 41)

3. As you might have guessed, we'll now want to set the Layer value of our Projectile's CollisionObject3D to 2. Double-click on projectile.tscn; then, with the Projcetile selected in the top left, deselect the '1' values

within the Layer and Mask sections of the Inspector menu's CollisionObject3D section. Next, select the '2' value within the Layer section.

4. Go back into your code editor. Within mnchar.h, add the following line right after `void shoot_projectile()` within the `public` section:

   void _on_projectile_detector_body_entered(Node3D *node);

   This function will allow us to react to a Projectile's collision with a Mnchar's Projectile_Detector component.

5. Next, within mnchar.cpp, add the following lines to the end of your `_bind_methods()` definition:

```
ADD_SIGNAL(MethodInfo("mnchar_hit",
                   PropertyInfo(Variant::STRING, "mnchar_id"),
                   PropertyInfo(Variant::STRING, "firing_mnchar_id")));

ClassDB::bind_method(D_METHOD("_on_projectile_detector_body_entered",
"node"),
                  &Mnchar::_on_projectile_detector_body_entered);
```

   Here, we're adding a signal (along with two arguments) as a property so that it can get accessed later on by main.cpp. We're also providing Godot with information about our `_on_projectile_detector_body_entered` function so that we can connect the Projectile_Detector's built-in `body_entered` signal to it.

   (Reference 33)

6. Add the following right below your `shoot_projectile()` function definition within mnchar.cpp:

```
void Mnchar::_on_projectile_detector_body_entered(Node3D *node) {
UtilityFunctions::print(
    "on_body_entered() just got called within mnchar.cpp.");
Projectile *node_as_projectile = Object::cast_to<Projectile>(node);
String firing_mnchar_id = node_as_projectile->get_firing_mnchar_id();
emit_signal("mnchar_hit", mnchar_id, firing_mnchar_id);
queue_free();
}
```

   (References 8, 38, and 43)

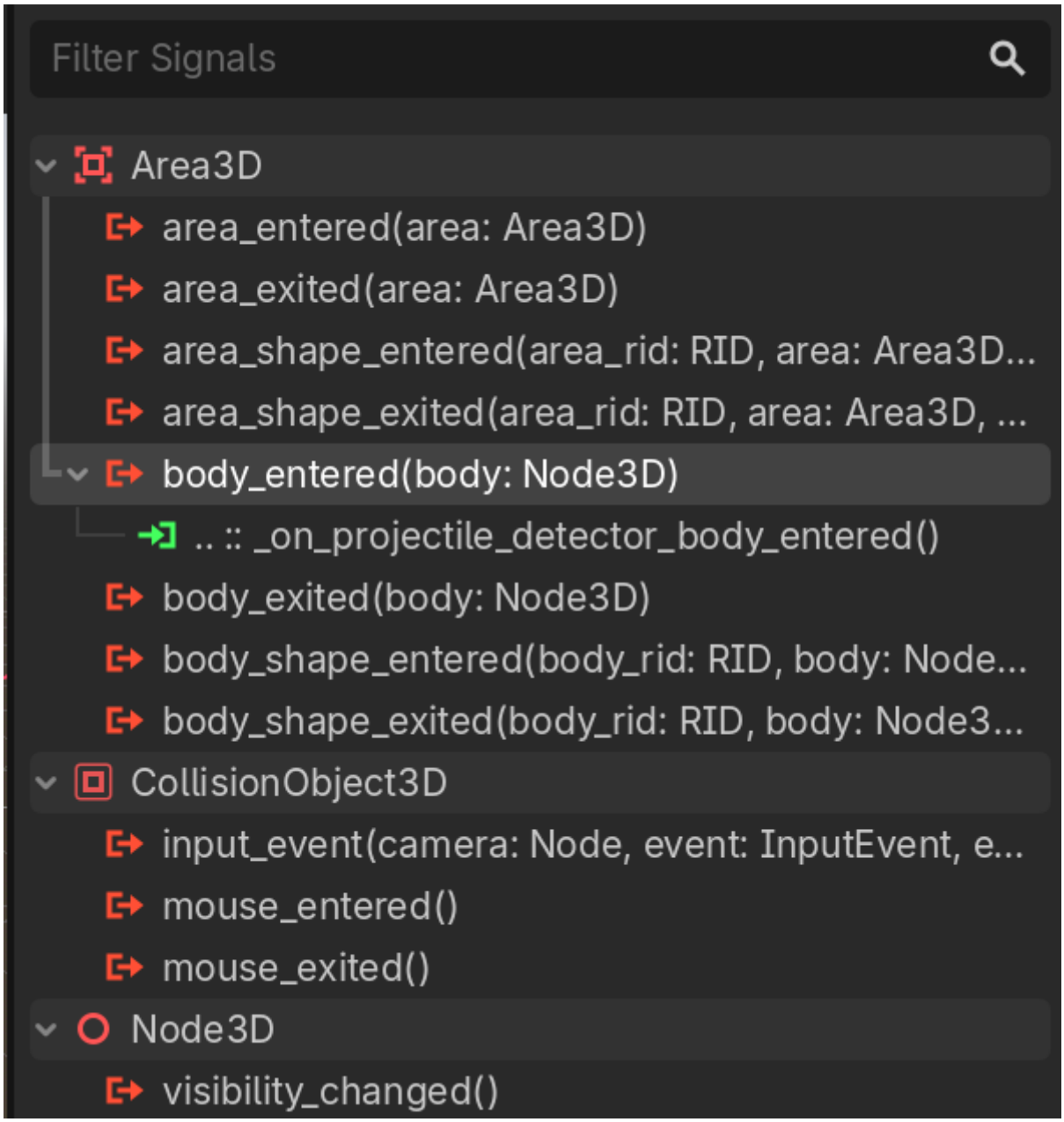
   The `queue_free()` function removes this Mnchar from the game.

Note that this function converts the `node` argument to a Projectile so that we can access the ID of its firing player. (This step would most likely cause the game to crash if items other than projectiles were registered by our Projectile_Detector. However, setting our layer and mask options correctly within the editor *should* ensure that only projectile collisions will cause this function to get called.)

7. Go ahead and compile the code, then restart the Godot editor to incorporate these changes. You'll find that, if you hit a Mnchar with a projectile, nothing will happen. This is because we haven't yet connected the Projectile_Detector's `body_entered` signal to our `_on_projectile_detector_body_entered` function.

8. There are two ways to do this. I'll first describe the GUI-based approach. Within the editor, select select the Projectile_Detector node within mnchar.tscn. Next, go to the Signals tab (to the right of the Inspector); right-click the `body_entered(body: Node3D)` signal; and select 'Connect'. Within the 'Connect a Signal to a Method' box that appears, Click on the Mnchar scene within the node list (which should bring up `_on_projectile_detector_body_entered` as a suggested function, then hit the 'Connect' button. This will inform the game that, whenever the `body_entered` signal gets triggered, the Mnchar's corresponding `_on_projectile_detector_body_entered` function should be called. (Reference 41)

   If all goes well, you should then see a little green icon appear below the `body_entered` signal in the Signals tab--along with the txt `.. :: _on_projectile_detector_body_entered()`. (The `..` signifies that you've conneced this signal to its parent, e.g. Mnchar.)



9. Relaunch your game and try firing a projectile at the green Mnchar. Once the projectile hits the Mnchar, it should now disappear.

10. This approach works fine, except I've found that connected signals sometimes disappear from the editor--which requires me to re-add them on occasion. (As with the disppearing-packed-scene behavior I mentioned earlier, I'm not sure what's behind this issue.) Therefore, I now prefer to add signals within code, which is *also* a great option for connecting signals between nodes that don't share a scene tree in the editor. (An example of this, which we'll get too shortly, is the connection of a Mnchar signal to Main. Since Mnchars aren't present within the Main class by default, we can't use the GUI to connect this signal--but we *can* do so using C++.)

11. To connect the Projectile_Detector's `body_entered` signal to `_on_projectile_detector_body_entered`, first add `#include`

`<godot_cpp/classes/area3d.hpp>` to the bottom of your main.h file's `#include` statements. Next, within main.cpp, add the following to the end of your `Mnchar::start()` function:

```
get_node<Area3D>("Projectile_Detector")->connect(
"body_entered", Callable(this, "_on_projectile_detector_body_entered"));
```

This code first retrieves the Mnchar's Projectile_Detector, then connects its body_entered signal to the `_on_projectile_detector_body_entered` function of 'this' (which refers to Mnchar). (Reference 1)

Note that it doesn't appear necessary to mention the Node3D argument of `body_entered` (listed within the Godot editor) within this `connect()` call.

If we didn't have a `start()` function, we could have added this within a new `_ready()` function instead. (In contrast, I found that attempting to add this code within the `Mnchar::Mnchar()` constructor did *not* work--perhaps because the Projectile_Detector wasn't yet accessible at that time.)

If you compile your code once again and restart the game, you should still be able to remove another Mnchar from the game by hitting it with a projectile, even if the editor doesn't show a connection between the Projectile_Detector's `body_entered` signal and the Mnchar's `_on_projectile_detector_body_entered` function.

Part 13: Ending the game

We're very close to having a working prototype of our game--one in which two players can attempt to hit one another with projectiles. Unless both players hit each other at the same time, we should be able to figure out who won (e.g. by seeing which player is left standing). However, it will be ideal to have the game determine this as well.

1. Within main.h, add the following code right below your `mnchar_id_color_dict` initialization:

   `HashSet<String> active_mnchars{};`

   This HashSet will store a list of active Mnchars. Once the size of this set becomes less than two, we can determine which Mnchar (if any) won the game. (If two Mnchars hit each other at exactly the same time, the length of this set will become 0, and no winner will be declared.)

   (References 32, 44, and 45)

2. Next, right below `add_child(new_mnchar)` within the for loop in main.cpp's `Main::_ready()` function, add:

   `active_mnchars.insert(mnchar_id_arg);`

   This way, every Mnchar in the `mnchars_to_include` array will also get added to our list of active players.

(I believe we could also have simply used the `mnchars_to_include` array to keep track of our active players; however, I did want to introduce the HashSet type within this tutorial, and this is a great way to do so.)

3. Finally, at the end of `Main::_ready()`, add the following code:

```
UtilityFunctions::print("Printing out all active players in set:");
for (auto active_mnchars_iterator = active_mnchars.begin();
    active_mnchars_iterator != active_mnchars.end();
    ++active_mnchars_iterator) {
    UtilityFunctions::print(*active_mnchars_iterator);
}
```

This both demonstrates how to iterate through a HashSet and helps identify which Mnchars got added to the game. (Reference 46)

4. Once we allow multiple games to be played within a single session, we'll want to clear out `active_mnchars` before each game so as to prevent dormant Mnchars from lingering around. To do so, add the following line at the start of `Main::_ready()`:

```
active_mnchars.clear();
```

5. In order to *remove* Mnchars from `active_mnchars` (and thus figure out when the game is over), we'll need to connect the `mnchar_hit` signal from main.cpp to a function within main.cpp that can process it. Let's declare and define this function now. First, within main.h, add the following right before `void _ready()` within the `private` section of main.h:

```
void _on_mnchar_mnchar_hit(String hit_mnchar_id_arg,
                           String firing_mnchar_id_arg);
```

6. Next, at the bottom of your `Main::_bind_methods()` function within main.cpp, add:

```
ClassDB::bind_method(D_METHOD("_on_mnchar_mnchar_hit",
"hit_mnchar_id_arg"),
                     &Main::_on_mnchar_mnchar_hit);
```

That way, we can reference this function within some signal-connection code that we'll add in shortly. (Note that both the function and its parameter should be included here.)

7. Finally, right above your `void Main::_ready() function` in main.cpp, define this new `Main::_on_mnchar_mnchar_hit` function as follows:

```
void Main::_on_mnchar_mnchar_hit(String hit_mnchar_id_arg,
                                 String firing_mnchar_id_arg) {
UtilityFunctions::print("The Mnchar with an ID of ", hit_mnchar_id_arg,
                        " was just hit by the Mnchar with an ID of ",
                        firing_mnchar_id_arg, ".");
active_mnchars.erase(hit_mnchar_id_arg);
// See godot-cpp/include/godot_cpp/templates/hash_set.hpp

UtilityFunctions::print("Current size of active_mnchars: ",
                        active_mnchars.size());

if (active_mnchars.size() == 1)

{
    String winning_mnchar = *active_mnchars.begin();
    UtilityFunctions::print("The player with the ID of "+
winning_mnchar+" won the game.");
}

else if (active_mnchars.size() == 0) {
    UtilityFunctions::print("Nobody won the game.")
}
}
```

If, after a Mnchar gets hit, only one Mnchar remains in the HashSet, we can identify the winning Mnchar by creating an iterator to the first (and only) item in this set (using `.begin()`), then dereferencing it to identify the winning ID.

Meanwhile, if the final two players got hit at the exact same time, `active_mnchars` will be empty. In this case, we'll announce that no one won the game.

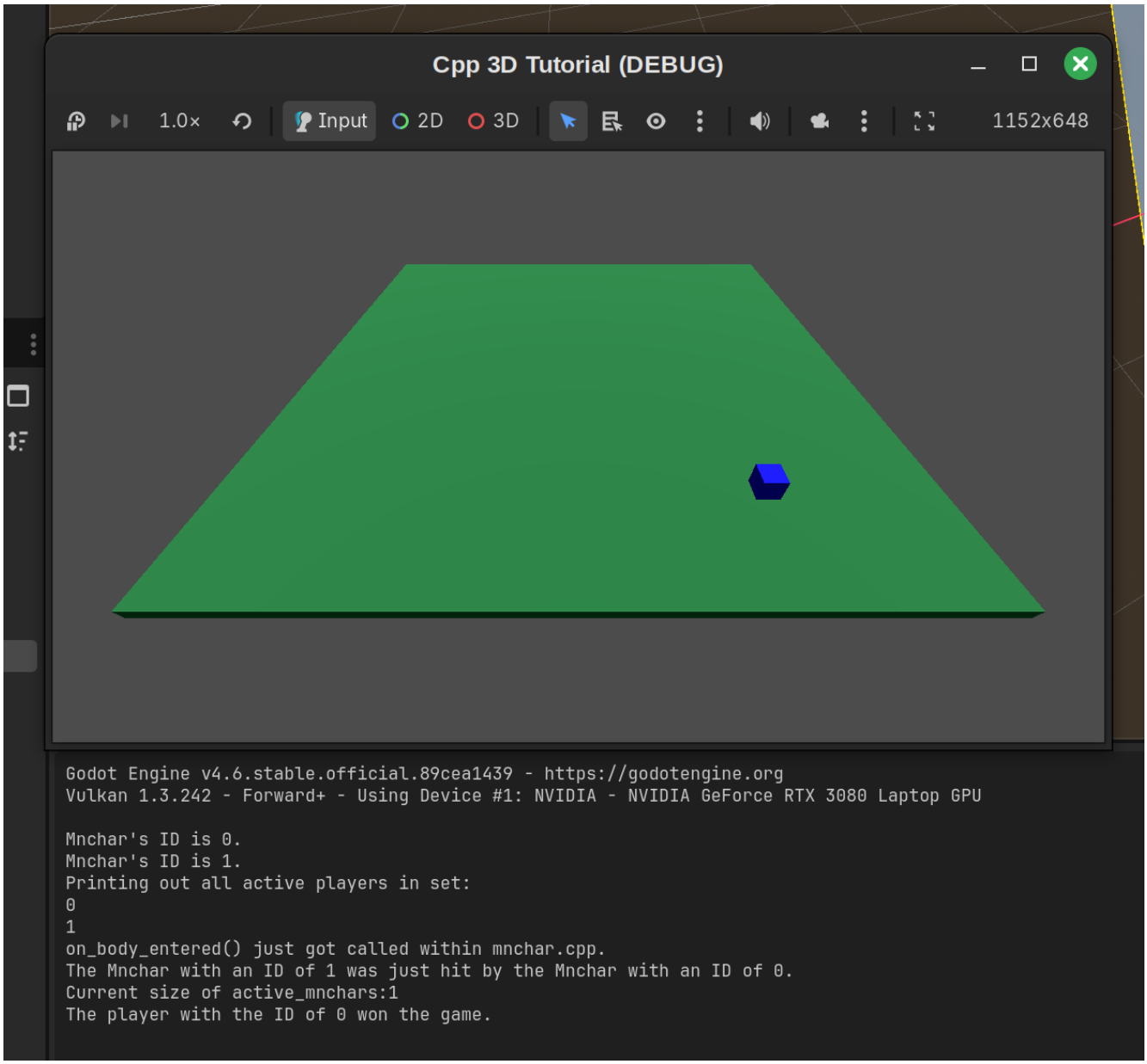
(References 45 and 46)

8. Next, add the following line right after `auto new_mnchar = reinterpret_cast<Mnchar *>(get_mnchar_scene()->instantiate());` within `Main::_ready()` in main.cpp:

```
new_mnchar->connect("mnchar_hit", Callable(this,
"_on_mnchar_mnchar_hit"));
```

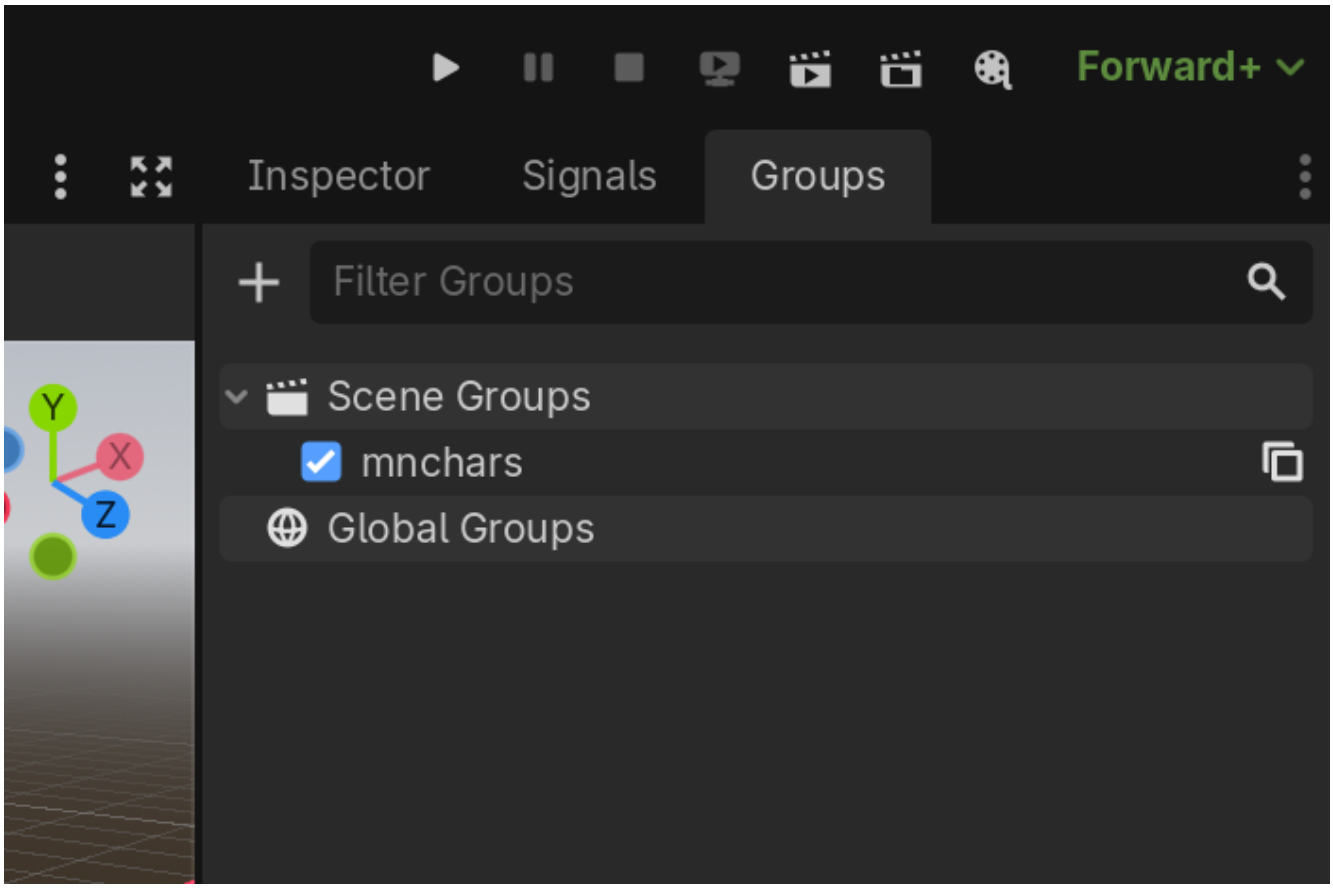
This will instruct Godot to call `_on_mnchar_mnchar_hit` whenever a Mnchar gets hit. (It's helpful, and perhaps necessary, to perform this connection via code rather than the editor, as the Mnchars in the game won't be present within the main.tscn scene tree beforehand.)

9. Compile your code, restart your editor, and try hitting the blue Mnchar with a projectile. Once you succeed, the console should inform you that "The player with the ID of 0 won the game."



10. When a game ends, we'll want to remove the final active player (e.g. the winner) from the scene. Otherwise, that player will still be active when we begin a new game--which would be an interesting mechanic for sure, but not one we'll want for this tutorial.

    To prepare to remove all players, select your Mnchar within mnchar.tscn's scene tree, then go over to the Groups tab (to the right of the Inspector and Signals tabs). Click the '+' to the left of the text bar to open up a Create New Group dialog. Name your group 'mnchars' and hit 'OK.'



11. We'll make use of this new scene group within a new `end_game()` function. Within main.h's `public` section, add `void end_game(String winning_mnchar_id);` right after your `void _on_mnchar_mnchar_hit()` function declaration. In addition, add `#include <godot_cpp/classes/scene_tree.hpp>` to the bottom to that file's list of `#include` statements.

12. Next, right before `void Main::_ready()` within main.cpp, define this new function as follows:

```
void Main::end_game(String winning_mnchar_id) {
get_tree()->call_group("mnchars", "queue_free");

String new_winner_message = "The winning player \
is: " + winning_mnchar_id + "\n\n";

UtilityFunctions::print(new_winner_message);
}
```

(Reference 8)

The first line of the function removes all Mnchars within the 'mnchars' group that we just created, thus preparing our game area for a subsequent round.

13. Update the `if/else if` section of `void Main::_on_mnchar_mnchar_hit())` so that it reads as follows:

```
if (active_mnchars.size() == 1)

{
String winning_mnchar = *active_mnchars.begin();
end_game(winning_mnchar);
}

else if (active_mnchars.size() == 0) {
end_game("Nobody");
}
```

Now that we're printing the winning player within `end_game()`, I removed the print() calls from this `if/else if` code, though you can also keep them in if you'd prefer.

14. Compile your code, restart Godot, and play your main scene. When you hit one Mnchar with another, both Mnchars should be removed from the game area.

Part 14: Adding a heads-up display (HUD) class

Congratulations! You now have a working 3D multiplayer game in C++. After launching the game, you and a friend or family member can try hitting each other's Mnchar with a projectile; once one of you succeeds, the console will announce a winner, and the game will end. You can then restart the scene to play again.

However, there are plenty of ways that we can improve on this setup:

- Since the debug console won't always be available, some in-game text should let you know who won.
- It would be ideal to be able to launch a new game without restarting the scene.
- Before we launch a new game, we should allow new players to join and previous players to opt out.
- The game could show statistics on how many hits each player achieved within the previous game--and the overall number of hits and wins each player has earned since the session began. There should also be a way to reset these overall stats.
- Players should be able to exit out of a game in progress while also preventing this canceled game from affecting their overall stats.

To implement these enhancements, we'll create a new Hud class that can display in-game text *and*, via its `_process` function, handle new-game-setup tasks. Here's a simplified overview of how this class will interact with Main:

- When `Main::_end_game()` is called, a label within Hud will announce the winner. Next, Hud's `_process()` function will launch, thus allowing new players to enter the game.
- Hud will store, and display, information about which players have chosen to enter the next game. (It will gather this information by checking for player inputs within `_process()`.)
- Once a player holds down both the `fire` and `reset` buttons, Hud() will emit a signal, along with information about the players who want to join, that Main will use to (1) call a new game-start function

and (2) pause Hud's `_process()` function.

- To begin setting up the Hud class, create two new files--'hud.h' and 'hud.cpp'--within your src/ folder. (Since Hud is the final class we'll add during our tutorial, these will be our final two source files.)

1. Within hud.h, add the following code:

```
#pragma once

#include <godot_cpp/classes/canvas_layer.hpp>
#include <godot_cpp/variant/typed_dictionary.hpp>

using namespace godot;

class Hud : public CanvasLayer {
GDCLASS(Hud, CanvasLayer)

private:
bool reset;
Size2 screenSize;

static void _bind_methods();

const String instructions = "To join this game, press Fire on \
your controller. To launch a game, press both Fire and Reset \
simultaneously. To reset overall stats, hold down Reset for \
two seconds.\n\n";

String winner_text{""};
String instructions_text{""};
String entrants_text{""};

Array mnchars_to_include {};

bool can_launch_new_game = true;

public:


Hud();
~Hud();

String get_instructions();

void set_winner_text(const String winner_arg);

void set_instructions_text(const String instructions_arg);

void set_entrants_text(const String entrants_arg);
```

```
void update_between_game_message();

void set_can_launch_new_game(bool can_launch_new_game_arg);

void _process(double delta) override;
};
```

Hud extends Godot's CanvasLayer class. It also defines a number of message components (`winner_message`, `instructions_message`, and `entrants_message`) that will serve as the building blocks of a between-game message that the HUD will display. Our approach will be to update these building blocks periodically, then call `update_within_game_message()` as needed to combine these message components together. This function will also display this combined message via Labels that we'll add within the editor.

(This surely isn't the only way to go about this task, but by presenting all of these messages together, we avoid having one message block partially cover another, and also decrease the amount of individual Label classes that we'll need to use.)

Similar code for updating a message that will get displayed *within* games will get added later.

By the way, the `_process` function declared here won't actually override Hud's process function without the `double delta` argument--so you should include it even if you don't actually need to make use of that parameter. (This may be obvious to many of you, but it did trip me up when I was first putting this code together, so I thought I would share it.)

The `can_launch_new_game` variable will be used to help ensure that a new game is not started multiple times in a row by our Hud's `_process()` function.

Note: when defining your `instructions` variable, don't add any space in between the start of lines 2 and onward and the text itself. Otherwise, these spaces will also show up within the in-game text. (See this project's hud.h file for reference if needed.)

(References 48, Reference 8)

2. Next, within hud.cpp, add:

```
#include "hud.h"

using namespace godot;

Hud::Hud() { reset = false; }
Hud::~Hud() {}

void Hud::_bind_methods() {}

String Hud::get_instructions()
```

```
{
return instructions;
}

void Hud::set_winner_text(const String winner_arg) { winner_text =
winner_arg; }

void Hud::set_instructions_text(const String instructions_arg) {
instructions_text = instructions_arg;
}

void Hud::set_entrants_text(const String entrants_arg) {
entrants_text = entrants_arg;
}

void Hud::update_between_game_message() {
String between_game_message =
    winner_text + instructions_text + entrants_text;
}

void Hud::set_can_launch_new_game(bool can_launch_new_game_arg) {
can_launch_new_game = can_launch_new_game_arg;
}

void Hud::_process(double delta) {}
```

Note how `update_between_game_message` combines the output of `winner_text`, `instructions_text`, and `entrants_text` into a single string. (Since this function performs the task of retrieving all three text blocks, it wasn't necessary to create separate `get` functions for them.)

(Reference 8)

3. Finally, go back into register_types.cpp in order to inform Godot of this new class. You can probably guess at this point which two lines to include, but in case you need a refresher: add `#include "hud.h"` below `#include "projectile.h"` near the top, and `GDREGISTER_RUNTIME_CLASS(Hud);` below `GDREGISTER_RUNTIME_CLASS(Projectile);` within `initialize_example_module()`.

Part 15: Configuring hud.tscn and extending the Hud class

1. Compile your code, then restart your editor. As you did with your other three scenes, click on Scene --> New Scene; select 'Other Node' within the 'Create Root Node' menu; choose your newly-created Hud class; and then save this scene as hud.tscn. (This will be the last scene that we add within this tutorial.)

2. Add a Label as a child of Hud and rename it 'BetweenGameLabel.' This label, as the name suggests, will display relevant text (e.g. the winner of the previous game, if applicable; instructions for starting a new game; and the players who have been added to the game) in between active games. (Reference 48)

3. Go to BetweenGameLabel's inspector, then expand the Transform section. Set the x and y Position values to 20.0 pixels each in order to create a small buffer between this box and the edge of the game window. Next, set the x Size value to 300.0 pixels. Scroll back up to the Label section, then set the Autowrap Mode to 'Word'. This way, we won't need to set line breaks manually within our message (though we could do so if needed using `\n`).

4. Double-click on main.tscn, then right click over 'Main' and select 'Instantiate Child Scene:.' Double click on your new hud.tscn file to add it to your scene tree.

5. We'll need to handle two different 'out-of-game' scenarios within the Hud: (1) the experience when you first launch the game, and (2) what you see after finishing a game. We'll handle the first scenario first, since it's a bit simpler. Open up main.h, then add the following line right before your `end_game()` function within the script's `public` section:

```
void _on_hud_start_game(Array mnchars_to_include);
```

6. In addition, add `#include "hud.h"` right below `#include "mnchar.h"` within this script's list of `#include` statements.

7. Next, right before your `Main::end_game()` function definition within main.cpp, add:

```
void Main::_on_hud_start_game(Array mnchars_to_include)
{
}
```

   Next, *cut* all of the contents between Main::_ready()'s opening and closing brackets, then *paste* them in between the starting and closing brackets of this new function. (Adding this code within Main::ready() allowed us to automatically launch each game with two players. However, we'll now want to allow code within Hud to specify when the game should be launched and how many Mnchars, exactly, should be included. Thus, it should now be placed within a new function.)

   As the name of this function suggests, we'll eventually connect it to a 'start_game' signal that Hud will emit.

8. Within `void Main::_ready()`'s opening and closing brackets, enter the following lines:

```
get_node<Hud>("Hud")->set_winner_text(
    "Welcome to Cube Combat!\n\n");

get_node<Hud>("Hud")->set_instructions_text(
get_node<Hud>("Hud")->get_instructions());

get_node<Hud>("Hud")->update_between_game_message();
```

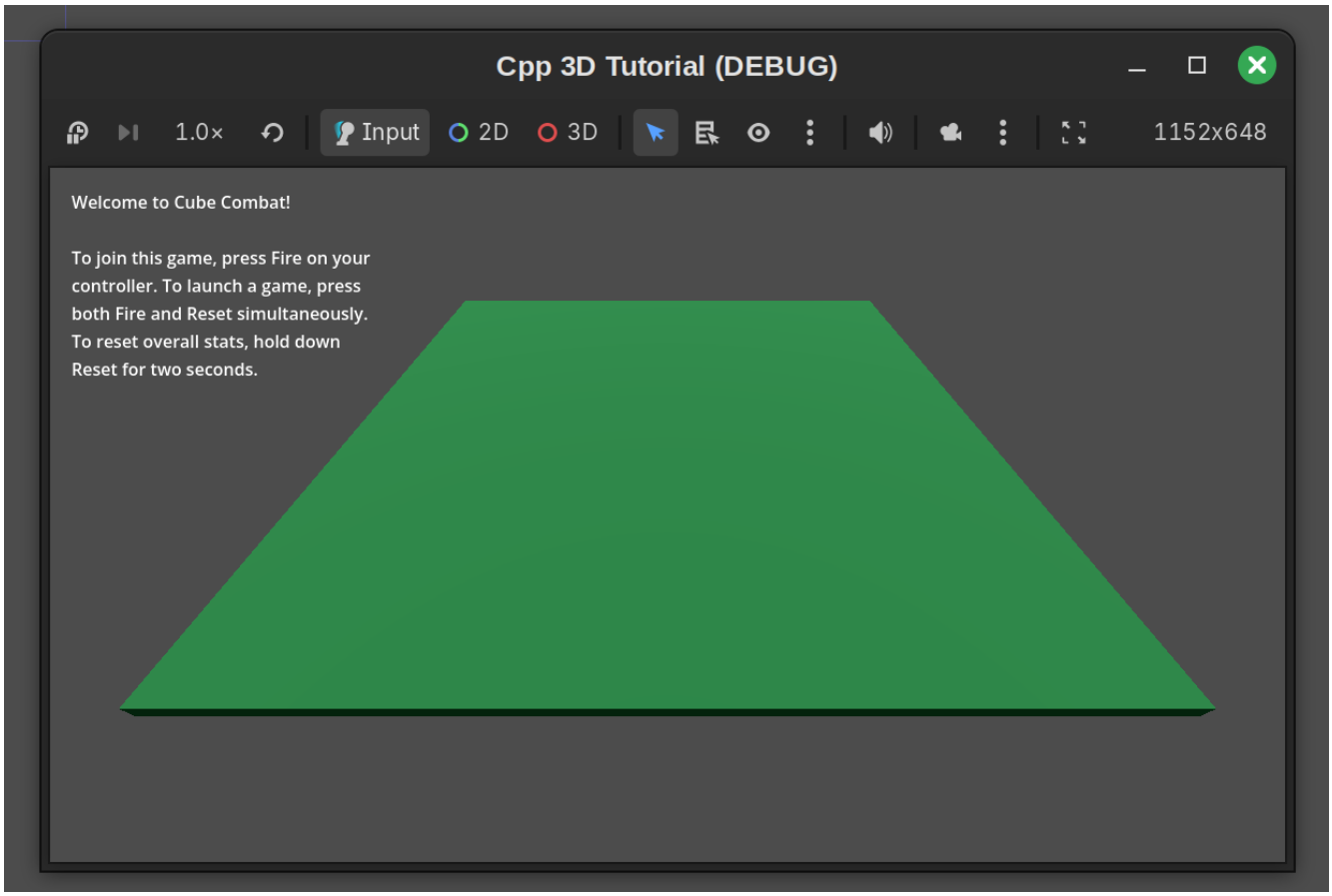
9. This code adds our standard game-instructions text to the Hud's `instructions_text` string, then instructs the Hud code to update our between-game message to include this new text. (We won't always want to display the gameplay instructions, so it's useful to be able to clear them out, which our current setup allows.) It also precedes these instructions with a welcome message. (We'd normally display information here about the previous game's winning player, but since we're just starting the game, we don't have any such game to reference.)

10. Go back to hud.h, then add `#include <godot_cpp/classes/label.hpp>` right below `#include <godot_cpp/variant/typed_dictionary.hpp>`. Next, within hud.cpp, add the following lines to the bottom of `Hud::update_between_game_message()`:

    ```
    auto between_game_label = get_node<Label>("BetweenGameLabel");
    between_game_label -> set_text(between_game_message);
    ```

    (Reference 8)

11. Compile your game, restart your editor, and launch your main scene. The BetweenGameLabel should now display the `between_game_message` you just configured in the top right.



12. These instructions aren't correct yet, since we haven't added the corresponding code that will make them valid. Let's work on adding that code now. First, within hud.h, add `#include`

`<godot_cpp/classes/input.hpp>` to the bottom of your list of `#include` statements. Next, fill in your `Hud::_process()` function within hud.cpp with the following code:

```
auto input = Input::get_singleton();

for (int i = 0; i < 8; i++)

    {

        String strint = String::num_int64(i);
        if (
        (input->is_action_just_pressed("fire_" + strint)) &&
        (mnchars_to_include.has(strint) == false)
        )

        {
        UtilityFunctions::print("Adding player " + strint + " to the
game.");
        mnchars_to_include.append(strint);

        }

        if ((input->is_action_pressed("fire_" + strint)) &&
            (input->is_action_pressed("reset_" + strint)) &&
        (can_launch_new_game == true))
            {UtilityFunctions::print("New game requested. Players:",
            mnchars_to_include);

            instructions_text = "";
            winner_text = "";
            entrants_text = "";
            update_between_game_message();
            can_launch_new_game = false;
            emit_signal("start_game", mnchars_to_include);
            }
}
```

This loop checks for relevant inputs from all eight possible players (with corresponding IDs of 0 through 7), then responds accordingly. When players request to get added to the game (by pressing their respective Fire buttons), they'll get added to the `mnchars_to_include` array--provided their IDs aren't already present within there.

Once a player presses both the Fire and Reset button, the between-message text will get cleared (since we won't need this message when a game is in progress), and a "start_game" signal will get emitted. However, these steps will only be taken if our `can_launch_new_game` bool is set to true.

(References 8 and 49)

13. Within Hud::_bind_methods() {} in hud.cpp, add:

```
ADD_SIGNAL(MethodInfo(
    "start_game", PropertyInfo(Variant::ARRAY, "mnchars_to_include")));
```

This signal will soon get processed by main.cpp's `_on_hud_start_game` function. By the way, if you need to check how to specify a given Variant within the PropertyInfo section of an ADD_SIGNAL() call, you can check the godot-cpp code's variant.hpp file (reference 50), which includes types like Variant:ARRAY and Variant:DICTIONARY.)

(References 1 and 50)

14. Relaunch your game, then press keys 1 through 7 on your keyboard followed by the Space Bar. (These were configured within your input settings to represent 'fire' actions for each possible player. Use the keys above your letters rather than the number pad.) Next, press both the Space Bar and R to request a new game. This should produce output like the following within your console:

```
Adding player 1 to the game.
Adding player 2 to the game.
Adding player 3 to the game.
Adding player 4 to the game.
Adding player 5 to the game.
Adding player 6 to the game.
Adding player 7 to the game.
Adding player 0 to the game.
New game requested. Players:["1", "2", "3", "4", "5", "6", "7", "0"]
```

15. We'll now allow the new-game request within `Hud::_process()` to actually start a game. Within main.cpp, add the following to the start of `Main::_ready()`:

```
get_node<Hud>("Hud")->connect("start_game",
    Callable(this, "_on_hud_start_game"));
```

(This could also be accomplished within the Godot editor.)

Note that, while we needed to include the `mnchars_to_include` argument within our recent ADD_SIGNAL() edition within main.h, it doesn't need to be mentioned within this `connect()` function call.

(Reference 1)

16. Next, at the end of `Main::_bind_methods()` within main.cpp, add:

```
ClassDB::bind_method(D_METHOD("_on_hud_start_game"),
                     &Main::_on_hud_start_game);
```

(Without this addition, your Hud class's `start_game` signal won't actually call your Main class's `_on_hud_start_game()` function.)

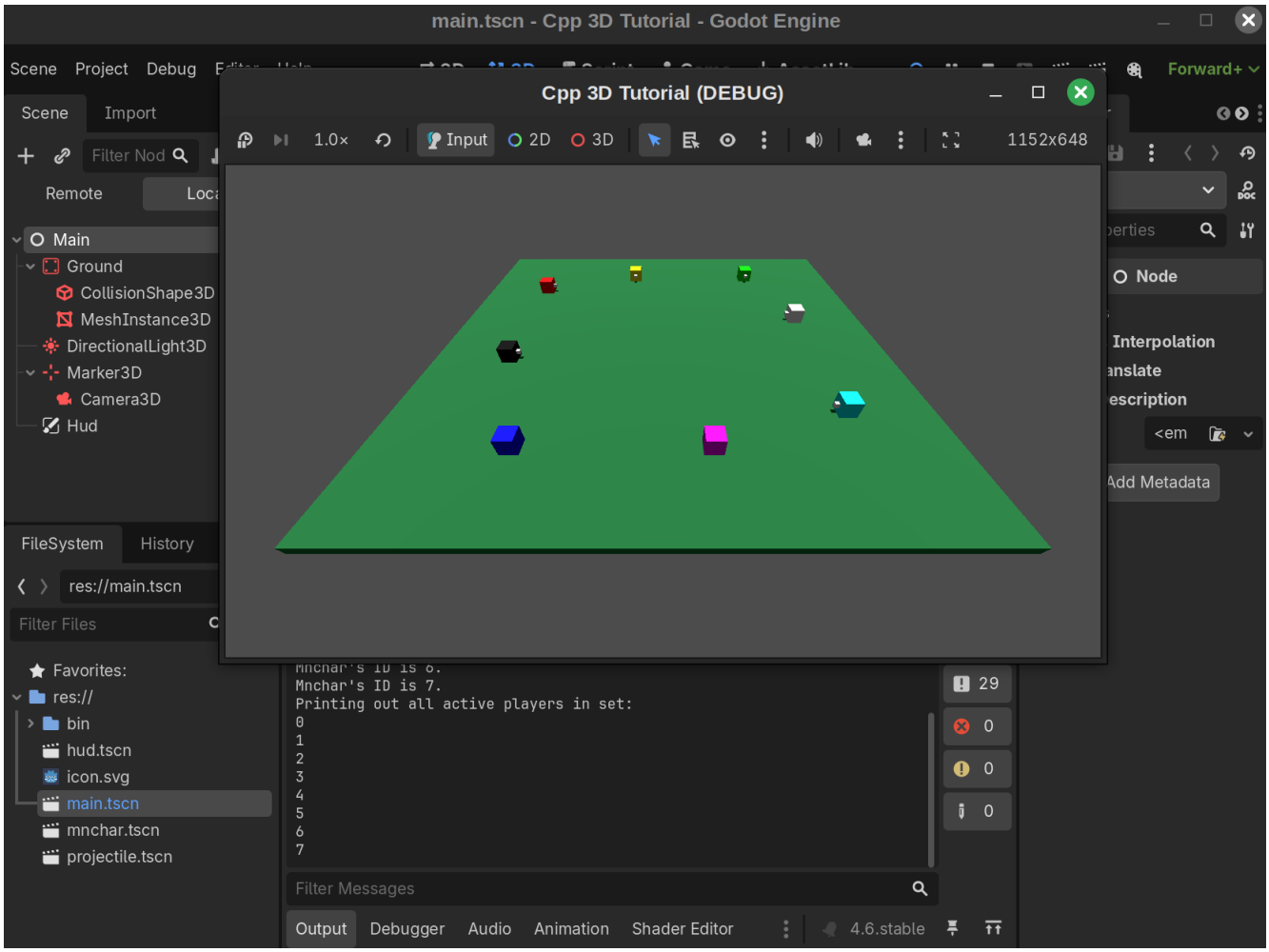
17. Finally, at the very beginning of `Main::_on_hud_start_game()`, add the following code:

```
get_node<Hud>("Hud")->set_process_mode(PROCESS_MODE_DISABLED);
```

This line stops Hud's `_process` script from running. That way, within-game player actions won't have any effect on that script. (Alternatively, if we needed `_process` to continue running, we could have added a `can_launch_new_game` flag that would instruct the `_process` script to skip past the loop we specified earlier. See my hud.cpp code within Reference 2 for an example. This tutorial's code, though, is far more readable and less cluttered than that version.)

(References 51 and 52)

18. Compile your code, restart your editor, and launch your game. Press keys 1-7 on your keyboard, along with the space bar, to add all 8 Mnchars to your game; next, press Space Bar and R at the same time to launch it. You should now see the following:



Part 16: Handling end-of-game tasks

We can now launch our game with an arbitrary number of players. However, we also need to allow players to return to the game-setup menu once a game is complete.

1. First, within hud.h, add `void clear_mnchars_to_include();` right before `void _process()` within this script's `public` section. Next, right before `void Hud::_prcocess()` within hud.cpp, define this new function as follows:

   ```
   void Hud::clear_mnchars_to_include() {
   mnchars_to_include.clear();}
   ```

   This function resets the `mnchars_to_include` array, thus allowing players to leave the game before a new round begins.

   (Reference 49)

2. Within the Godot editor, add a Timer as a child of Main and name it 'HudProcessTimer.' We'll use this timer to give players time to review the winner of the previous game before restarting Hud's `_process()` function. (This timer will also prevent players from accidentally adding themselves to a new

game in the process of finishing their previous one, since the fire and player-addition actions will use the same button.)

Within the HudProcessTimer's Inspector, set the Wait Time to 2 seconds and the One Shot option to on. (The latter will prevent the timer from restarting after it counts down to 0.)

(By the way: I had originally made this timer a member of Hud. However, I wasn't able to get that code to work--perhaps because, when the Hud node was disabled, timer-related code wouldn't work correctly.)

(Reference 48)

3. In main.h, add `#include <godot_cpp/classes/timer.hpp>` to the end of your list of `#include` statements.

4. In main.cpp, add the following text to the bottom of `Main::end_game()`:

```
get_node<Hud>("Hud")->set_winner_text(new_winner_message);
get_node<Hud>("Hud")->update_between_game_message();
get_node<Timer>("HudProcessTimer")->start();
```

This code allows players to see the ID of the winning Mnchar within the UI. (We'll add updates later on to make this ID easier to interpret.) In addition, it begins the two-second timer that we just created within our Main scene.

If you'd like, you can also remove the `UtilityFunctions::print(new_winner_message);` line, since it's now redundant.

(References 53 and 54)

5. Back within main.h, insert `#include <godot_cpp/classes/timer.hpp>` at the end of your list of `#include` statements. Next, add `void _on_hud_process_timer_timeout();` right before `void _ready()` within this file's `public` section.

6. Within main.cpp, add the following code right before `Main::_ready()`:

```
void Main::_on_hud_process_timer_timeout() {
get_node<Hud>("Hud")->clear_mnchars_to_include();
get_node<Hud>("Hud")->set_instructions_text(
get_node<Hud>("Hud")->get_instructions());
get_node<Hud>("Hud")->update_between_game_message();
get_node<Hud>("Hud")->set_can_launch_new_game(true);
get_node<Hud>("Hud")->set_process_mode(PROCESS_MODE_ALWAYS);
}
```

`_on_hud_process_timer_timeout` resets the Hud's list of players to include; makes the instructions text visible again; and reactivates the Hud class's `_process` function. It also sets the Hud class's `can_launch_new_game` variable back to true so that a player can once again launch a new game by holding down the Fire and Reset buttons.

(Note: The code may very well function fine without the `can_launch_new_game` variable. I added it in just in case the Hud class's `_process` function might continue to emit a `start_game` signal before that function gets shut down by the Main class.)

7. Next, add the following code at the very start of `Main::_ready()`:

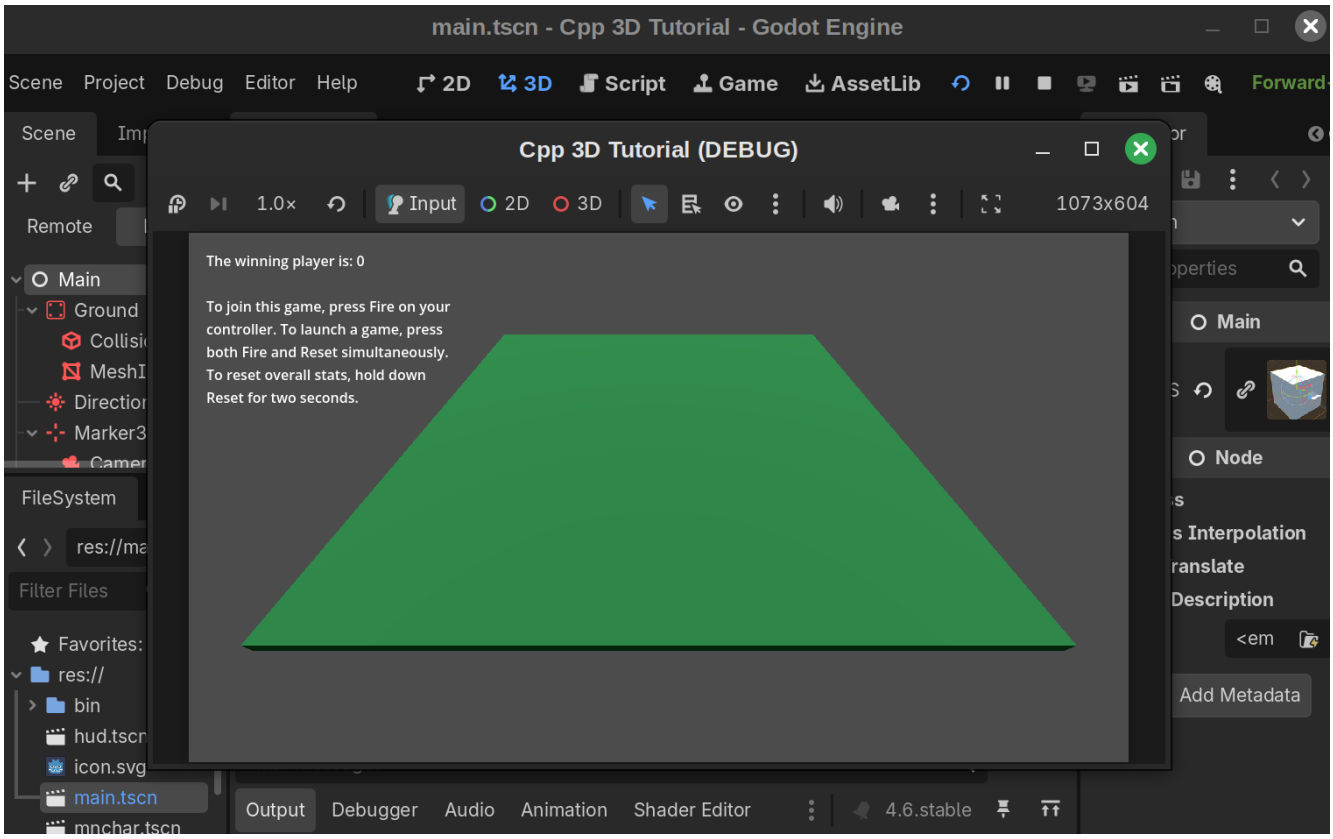
```
void Main::_ready() {
get_node<Timer>("HudProcessTimer")->connect(
"timeout", Callable(this, "_on_hud_process_timer_timeout"));
}
```

This code addition connects the timer's built-in `timeout()` signal to the `_on_hud_process_timer_timeout` function. (This `timeout()` signal can be found within the Signals tab when the HudProcessTimer is selected within the Godot editor.)

8. We'll also need to register `_on_hud_process_timer_timeout` within main.cpp's `_bind_methods()` function so that the signal-connection code within `_ready()` works. You can do so by adding the following code to the bottom of this function:

```
ClassDB::bind_method(D_METHOD(
"_on_hud_process_timer_timeout"), &Main::_on_hud_process_timer_timeout);
```

9. Compile your code, restart the editor, and launch your game. Try adding some players to the game, then clear all but one player out in order to end it. Once the game ends, you should see the a message about the winning player (but no other text). *Then*, after 2 seconds, you should see the game's instructions and regain the ability to add players.



We're not far from finishing the game at this point! However, we still need to add a few more UI elements, including running overall stats totals--and allow players to reset both those stats totals and active games.

Part 17: Further expanding our UI

Right now, we're referring to players using integers. However, these numbers won't make much sense to players on their own. Therefore, we'll add in a dictionary that allows players' colors to be displayed together with their IDs.

1. First, at the end of your `private` section within hud.h, add the following code:

```
    const TypedDictionary<String, String> mnchar_id_color_name_dict{
    {String("0"), "Blue"},  {String("1"), "Green"},   {String("2"),
"Cyan"},
    {String("3"), "Red"},   {String("4"), "Magenta"}, {String("5"),
"Yellow"},
    {String("6"), "White"}, {String("7"), "Black"}};
```

This dictionary clarifies which player color refers to which ID. (Each ID's color name should of course match its corresponding color value within the Main class's `mnchar_id_color_dict`.)

2. In addition, right after `~Hud();` within hud.h's `public` section, add:

```
TypedDictionary<String, String> get_mnchar_id_color_name_dict();
```

This will allow the Main class to retrieve this new dictionary. (We could also simply have made this dictionary public.)

3. Within hud.cpp, add the following function right before `String Hud::get_instructions()`:

```
TypedDictionary<String, String> Hud::get_mnchar_id_color_name_dict() {
return mnchar_id_color_name_dict;
}
```

(I had originally forgotten to add in this definition, which prevented Godot from successfully retrieving my classes within the editor. Adding the definition back in, compiling my code, and restarting Godot solved this issue.)

4. Still within hud.cpp, add the following right after `mnchars_to_include.append(strint)` within `Hud::_process()`'s first `if` statement:

```
entrants_text += "Added Player " + strint + " (" +
                 String(mnchar_id_color_name_dict[strint]) +
                 ") to the game.\n";
update_between_game_message();
```

This will not only help players link IDs and colors, but also notify them when a particular player has been added to the game.

5. We can also use this ID-color name dictionary to improve our game-winner message. First, though, we'll need to give main.cpp access to it. One option would be to copy and paste its definition into main.h, but this would then double the work needed to maintain it. Instead, simply add the following to the end of the `private` section of your main.h code:

```
TypedDictionary<String, String> mnchar_id_color_name_dict;
```

6. Next, open up main.cpp. Add the following to the very start of `Main::_ready()`:

```
mnchar_id_color_name_dict =
    get_node<Hud>("Hud")->get_mnchar_id_color_name_dict();
```

We're simply initializing the Main class's ID-color name dictionary as a copy of the Hud class's. This way, any changes we make to the Hud class's dictionary will automatically get applied within our Main class as well.

7. Within main.cpp, go to your `Main::end_game()` function. Replace the following code:

```
String new_winner_message = "The winning player \
is: " + winning_mnchar_id + "\n\n";
```

With the following:

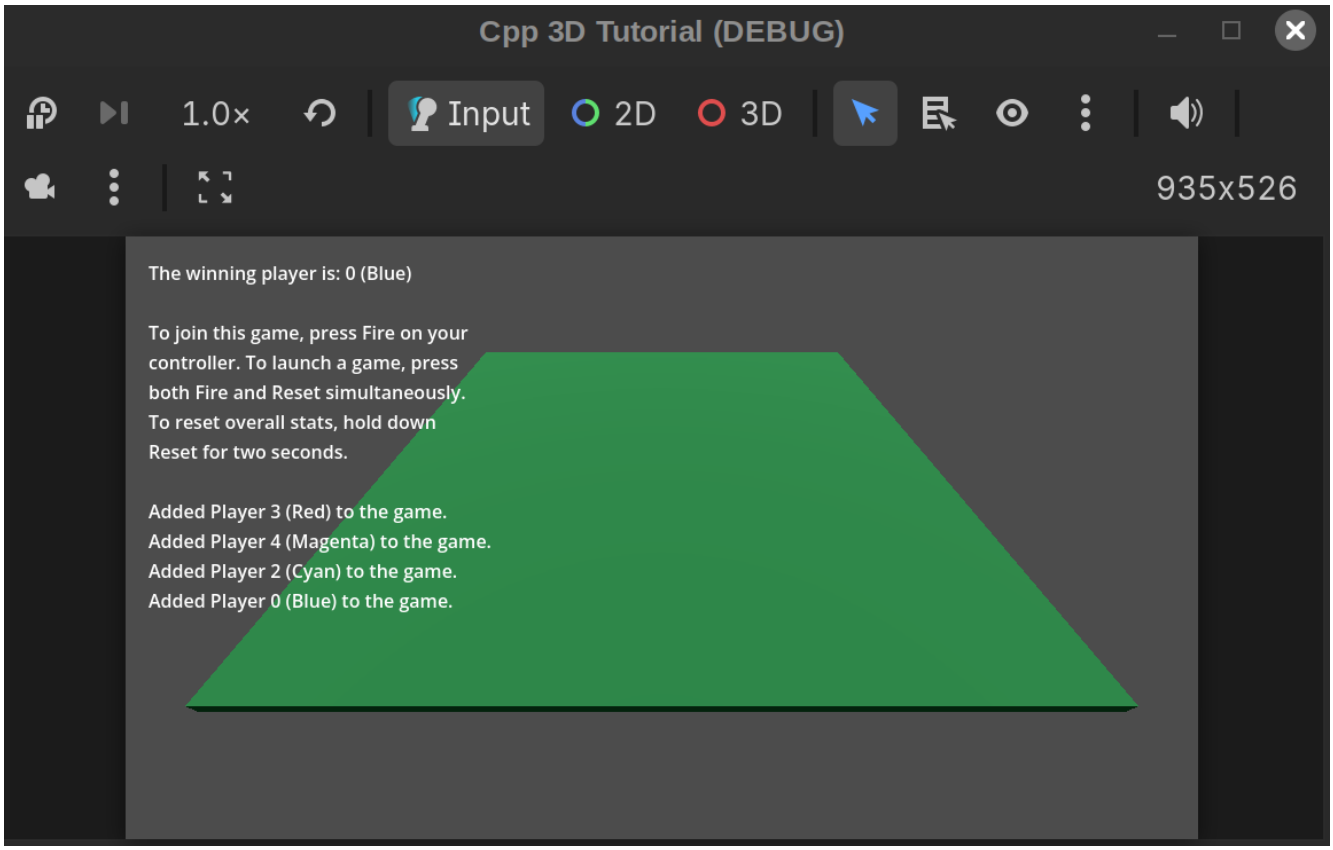
```
String new_winner_message = "The winning player \
is: " + winning_mnchar_id;

if (winning_mnchar_id != "Nobody") {
    new_winner_message +=
        " (" + String(mnchar_id_color_name_dict[
winning_mnchar_id]) + ")";
}

new_winner_message += "\n\n";
```

This code accounts for the possibility that two players will hit each other at exactly the same time, in which case no one will be the winner.

8. Compile your code, restart the Godot editor, and launch the game. You should now be able to see a list of entrants within the text that appears prior to the start of a new game. (This list will also get updated automatically whenever a new player gets added. In addition, the colors corresponding to these entrants' IDs, along with the winner's ID, will now be present within the between-game display.)



9. So far, the only 'stat' we're sharing with players is who won each game. It would be interesting, however, to keep track of--and display--the number of hits each player scored within each game, along with the overall number of hits and wins across games. Now that we have a decent amount of Hud code in place, this will be relatively easy to implement. First, add the following code to the bottom of the `private` section of main.h:

```
TypedDictionary<String, int> hits_achieved{};

TypedDictionary<String, int> overall_hits_achieved{};

TypedDictionary<String, int> overall_wins{};
```

These dictionaries will store the three stats mentioned above.

10. Next, open up main.cpp. Directly below `active_players.clear();` within `Main::_on_hud_start_game()`, add:

```
hits_achieved = TypedDictionary<String, int>{};
```

This will reset any data present within this dictionary, thus ensuring that only stats for the current game are contained within it.

11. Next, right after `active_mnchars.insert(mnchar_id_arg);` within this same function, add the following code:

```
hits_achieved[mnchar_id_arg] = 0;

if (overall_hits_achieved.has(mnchar_id_arg) == false)

{
  overall_hits_achieved[mnchar_id_arg] = 0;
}

if (overall_wins.has(mnchar_id_arg) == false) {
  overall_wins[mnchar_id_arg] = 0;
}
```

This code checks to see whether the ID of the Mnchar that just got added to the game is present within our overall hit and win dictionaries. If it's not, it will get added to both with a starting value of 0.

(Reference 55)

12. Within your `Main::_on_mnchar_mnchar_hit()` function definition in main.cpp, add the following code above `active_mnchars.erase(hit_mnchar_id_arg);`:

```
int current_hit_value = hits_achieved[firing_mnchar_id_arg];
current_hit_value += 1;
hits_achieved[firing_mnchar_id_arg] = current_hit_value;

int current_overall_hit_value =
overall_hits_achieved[firing_mnchar_id_arg];
current_overall_hit_value += 1;
overall_hits_achieved[firing_mnchar_id_arg] = current_overall_hit_value;
```

This code updates both the `hits_achieved` and `overall_hits_achieved` dictionaries to reflect this new hit.

13. Right after `new_winner_message += "\n\n";` within `Main::end_game()` in main.cpp, add the following code:

```
Array hits_achieved_keys = hits_achieved.keys();

for (int key_index = 0; key_index < hits_achieved_keys.size();
key_index++)
```

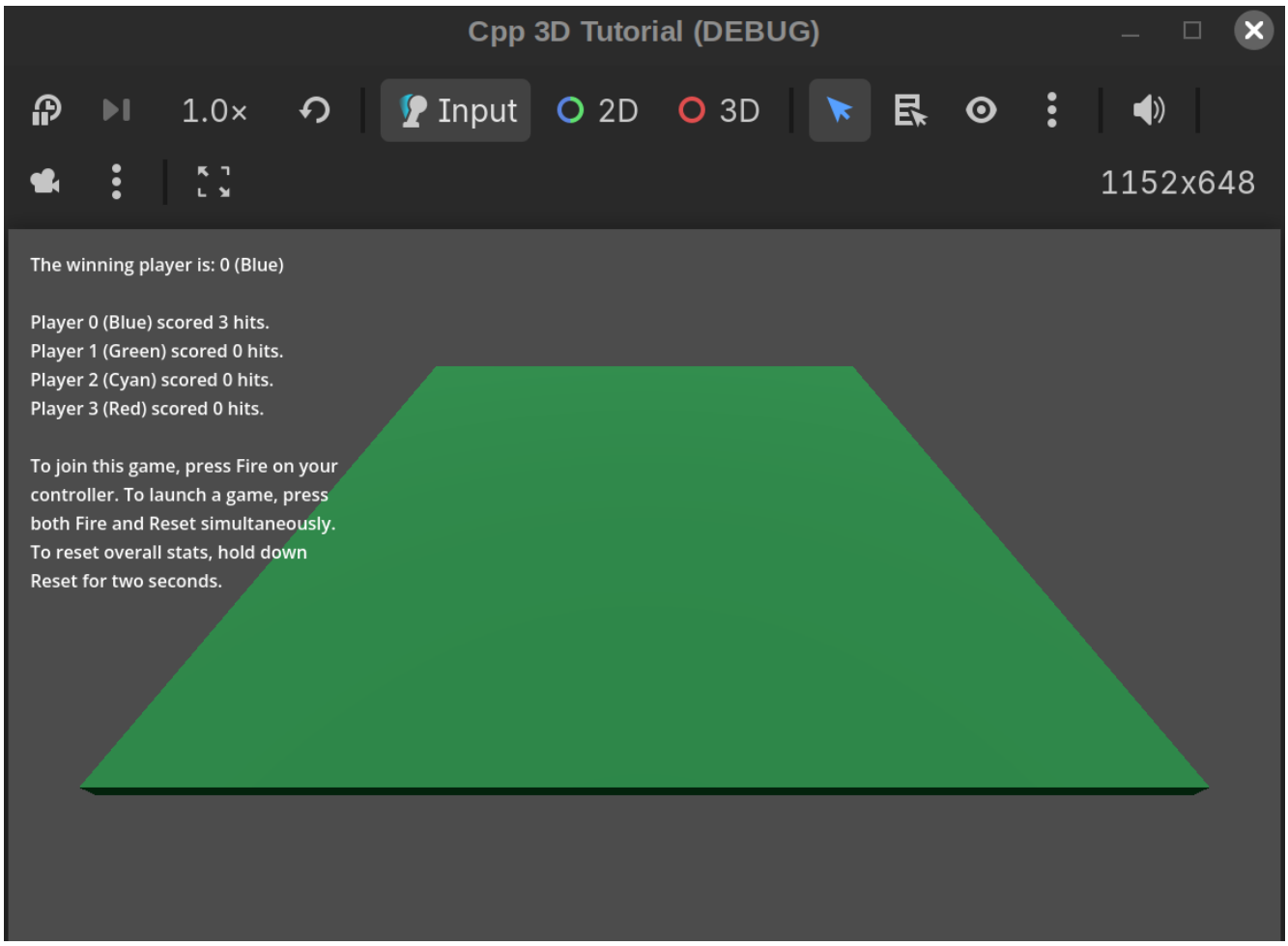
```
{
    String mnchar_id_arg = String(hits_achieved_keys[key_index]);
    String current_id_hits_achieved =
        String::num_int64(hits_achieved[mnchar_id_arg]);

    new_winner_message += "Player " + mnchar_id_arg + " (" +
                          String(mnchar_id_color_name_dict[mnchar_id_arg])
+
                          ")" + " scored " + current_id_hits_achieved +
                          " hits.\n";
}

new_winner_message += "\n";
```

Here, we're storing an array of all of the keys within `hits_achieved` so that we can more easily loop through this dictionary. For each of these keys, we'll determine how many hits the corresponding player scored, then add these to our new_winner_message. (This method of looping through a TypedDictionary is based on Reference 56.)

14. Compile your code, restart the editor, and launch the game. Once you've hit all other players with a given player, you should be able to see that player's stats within the between-game message screen:



    1. You may also want to verify that, if you launch a follow-up game with a new set of players, only those players (and not any who were removed after the first game) show up within these post-game hit stats.

15. While you're within the editor, Go ahead and add another Label child of Hud within hud.tscn. Rename it 'ConstantLabel'. Within the Inspector, set its Autowrap value to Word. We'll make use of this new label very soon.

16. This label will get displayed on the right side of the screen so as not to overlap with the between-game message. However, before we can set its specific location, we should first expand the game window to full-HD dimensions. Go to Project --> Project Settings, then select the 'Display' section within the menu on the left. Change the Viewport Width and Viewport Height options within the Window menu to 1920 and 1080, respectively.

    (You're welcome to choose smaller dimensions in order to allow your game to display on smaller monitors; you'll just then need to modify certain label settings specified in this tutorial in order to make them compatible with these custom dimensions.)

17. Next, within the Layout section of the ConstantLabel's Inspector window, set the Size's x value to 250.0 pixels and the Position's x value to 1650.0 pixels. (1920 - 250 - 20 (the same buffer we're using for our BetweenGameLabel) equals 1650.)

18. Now that we have a larger game window, we should also increase our font sizes accordingly. Within the Theme Overrides section of the ConstantLabel inspector, click on Font Sizes, then set the Font Size value to 20 pixels.

19. Increase the Font Size of the BetweenGameLabel to 20 pixels as well using the method described above. In addition, within the Transform section of the BetweenGameLabel's Inspector view, change the window's size to 600 pixels. (This will provide enough room for all between-game text to get displayed, even when 8 players are active.)

20. Next, within main.cpp's `Main::_on_mnchar_mnchar_hit()` function, add the following code right before `end_game(winning_mnchar)` within the `if (active_mnchars.size() == 1)` condition:

```
int current_overall_win_value = overall_wins[winning_mnchar];
current_overall_win_value += 1;
overall_wins[winning_mnchar] = current_overall_win_value;
```

21. The game is now keeping track of our overall win and hit information as well, but players don't have any way of viewing those stats. We can resolve this by adding a new set of code to our Hud class that will let us show this information. First, right after within the `private` section of main.h, add:

```
String overall_hits_text{""};
String overall_wins_text{""};
```

22. Next, within the `public` section of this file, add the following code directly below :

```
void set_overall_hits_text(const String overall_hits_arg);
void set_overall_wins_text(const String overall_wins_arg);
```

23. Next, add `void update_constant_message();` right below `void update_between_game_message();` within this `private` section.

    The idea here is to store a 'constant' message (i.e. one that appears both within and between games') made up of overall-hit information and overall-win information. This information will be stored within Strings (`overall_hits_text` and `overall_wins_text`) that our `update_constant_message()` function can access.

24. Enter hud.cpp. Right after your `Hud::set_entrants_text()` definition, add the following code:

```
void Hud::set_overall_hits_text(String overall_hits_arg) {
overall_hits_text = overall_hits_arg;
}

void Hud::set_overall_wins_text(String overall_wins_arg) {
overall_wins_text = overall_wins_arg;
}
```

25. Next, after your `Hud::update_between_game_message()` definition, add:

```
void Hud::update_constant_message() {
String constant_message = overall_hits_text + overall_wins_text;
auto constant_label = get_node<Label>("ConstantLabel");
constant_label->set_text(constant_message);
}
```

26. Because determining what text to place within the overall-hits and overall-wins Strings will be somewhat involved, we'll create separate functions within main.cpp for this process. Add the following code to the end of `main.h`'s `public` section:

```
void generate_overall_hits_text();

void generate_overall_wins_text()
```

27. Next, at the end of main.cpp, add the following definitions for these new functions:

```
void Main::generate_overall_hits_text() {
String overall_hits_text = "Overall hits:\n";

Array overall_hits_achieved_keys = overall_hits_achieved.keys();

for (int key_index = 0; key_index < overall_hits_achieved_keys.size();
    key_index++) {
    overall_hits_text +=
        "Player " + String(overall_hits_achieved_keys[key_index]) + " ("
+
        String(mnchar_id_color_name_dict[String(
            overall_hits_achieved_keys[key_index])]) +
        "): " +
        String::num_int64(
            overall_hits_achieved[overall_hits_achieved_keys[key_index]])
+
        "\n";
}

get_node<Hud>("Hud")->set_overall_hits_text(overall_hits_text);
get_node<Hud>("Hud")->update_constant_message();
}

void Main::generate_overall_wins_text() {
Array overall_wins_keys = overall_wins.keys();

String overall_wins_text = "\nOverall wins:\n";

for (int key_index = 0; key_index < overall_wins_keys.size();
key_index++)

{
    overall_wins_text +=
        "Player " + String(overall_wins_keys[key_index]) + " (" +
        String(mnchar_id_color_name_dict[String(
            overall_wins_keys[key_index])]) +
        "): " +
String::num_int64(overall_wins[overall_wins_keys[key_index]]) +
        "\n";
}

get_node<Hud>("Hud")->set_overall_wins_text(overall_wins_text);
get_node<Hud>("Hud")->update_constant_message();
}
```

These functions iterate through the overall_hits_achieved and overall_wins dictionaries to determine the number of hits and wins, respectively, achieved by each player during all games played thus far. They then update Hud's `overall_hits_text` and `overall_wins_text` values with this data, then call `update_constant_message()` to add these stats to the game window.

(We could also have had these new functions *return* their text values, then call `set_overall_hits_text()`, `set_overall_wins_text()`, and `update_constant_message()` separately.)

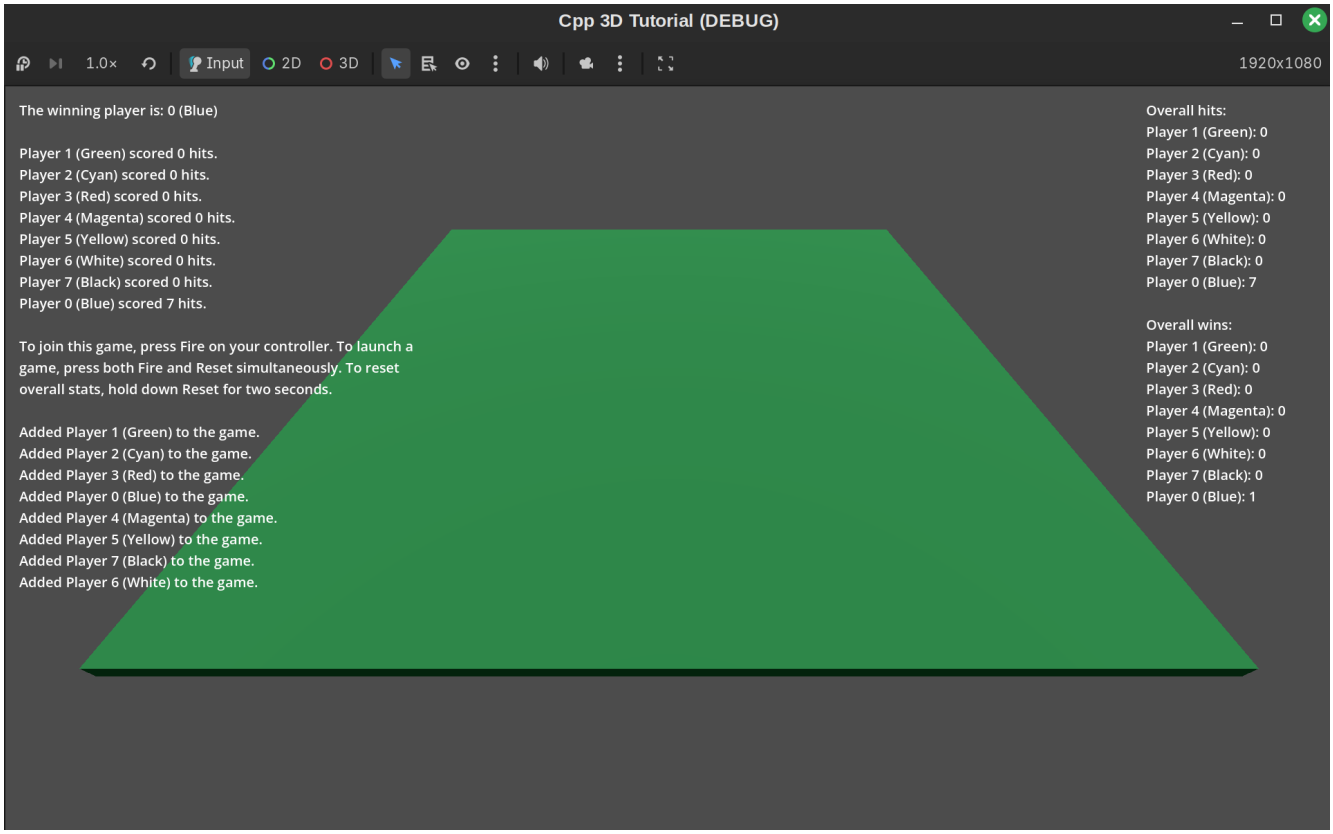
28. The final step is to add in calls to these two new functions where needed. First, after `active_mnchars.erase(hit_mnchar_id_arg);` within `Main::_on_mnchar_mnchar_hit()`, add `generate_overall_hits_text();`. Next, right *before* `end_game(winning_mnchar);` within this same function, add `generate_overall_wins_text();`.

29. Finally, at the very end of `Main::_on_hud_start_game`, add:

```
generate_overall_hits_text();
generate_overall_wins_text();
```

    This code will cause `update_constant_message()` to get called two times in quick succession. If we had excluded that function from `generate_overall_hits_text()` and `generate_overall_wins_text()`, we could then just call it once here. However, the current approach, while not the most efficient possible, shoudn't be too much of a computational burden.

30. Compile your code, restart the Godot editor, and launch your game. Try adding 8 players to the game, then hit seven of them with one of your players. Next, within the ensuing between-game menu, try adding another set of 8 players to the game. Your Hud text should appear as follows:



Although a larger font size would have been useful, the 20-pixel size we chose allows all of this text to fit within the screen, *and* prevents the constant text from overlapping with the game area, which would be a major distraction for players.

Part 18: Adding in reset options

In certain cases (and not just because they lost!), players may wish to exit out of an active game. For example, it's currently possible to add just one player to a game; this would result in a 'softlock' (https://en.wiktionary.org/wiki/softlock), as there's no way to exit out of this round without shutting down the game.

In addition, players might want to reset the hit and win stats on occasion. (For instance, players might want to do one or more practice rounds, then reset the statistics so that such rounds don't count towards their overall score.)

This part of the tutorial will add both of these features to the game using the Reset button that we've already added to our input map.

Here with editing:

Allow players to reset overall stats *and* games (while also removing the stats of reset games from your overall hit stats.) This will involve creating a few additional dictionaries.

1. Create Part 19: Final enhancements. (1) add boundaries to the game area so players can't fall off (make sure to update your layer/mask settings accordingly), *and* (2) update projectile colors to match those of their firing Mnchar.

Future editing notes

1. Try to make as many items `const` as possible.

References

Notes:

- In some cases, a reference within this list was itself based heavily (or even entirely) on another reference. However, in order to keep this list simple and manageable, I won't include such details. You can find more information about the sources used for these references within their own repositories.
- In addition, references that begin with '/godot-cpp' refer to a file within a *compiled* godot-cpp repository. (See Part 1 for more details on (1) how to access and compile this repository and (2) the exact version I'm using.)
- Not all references are listed within every paragraph. For instance, if multiple paragraphs in a row use the same reference, I might only cite that reference within some of those paragraphs.
- Reference 1: https://docs.godotengine.org/en/4.6/tutorials/scripting/cpp/gdextension_cpp_example.html
- Reference 2: https://github.com/kburchfiel/godot_cpp_3d_demo/
    - Since this repository is essentially a step-by-step tutorial to creating the code in Reference 2, both source files correspond very closely to one another. For example, the mnchar.h code within this repository was based mostly on https://github.com/kburchfiel/godot_cpp_3d_demo/blob/main/src/mnchar.h .
- Reference 3: https://docs.godotengine.org/en/4.6/getting_started/first_3d_game/01.game_setup.html
- Reference 4: https://docs.godotengine.org/en/4.6/getting_started/first_3d_game/03.player_movement_code.html
- Reference 5: https://docs.godotengine.org/en/4.6/getting_started/first_3d_game/02.player_input.html
- Reference 6: /godot-cpp/gen/src/classes/character_body3d.cpp
- Reference 7: /godot-cpp/gen/include/godot_cpp/classes/character_body3d.hpp
- Reference 8: https://github.com/kburchfiel/cpp_yf2dg_gd_4pt_6
    - The vast majority of the code in this repository came from https://github.com/j-dax/gd-cpp , which was released under the BSD-3-Clause license by Matthew Piazza. My repository simply converted that code into a step-by-step guide (similar to this one one).

    - In order to simplify this reference list, I won't link to all of the individual source files that I consulted. However, I recommend checking player.cpp and player.h for the Mnchar class's code; main.cpp and main.h for the Main class's code; and hud.cpp and hud.h for the Hud class's code. (These source files are available within https://github.com/kburchfiel/cpp_yf2dg_gd_4pt_6/tree/main/src .)

- Reference 9: https://godotengine.org/releases/4.3/#gdextension-runtime-class-registration
- Reference 10: https://forum.godotengine.org/t/gdextension-register-runtime-class/77868/4?u=kburchfiel
- Reference 11: https://www.reddit.com/r/godot/comments/13ikz4u/best_way_to_handle_controller_input_for_local/
- Reference 12: https://github.com/remram44/godot-multiplayer-example
- Reference 13: https://www.gdquest.com/library/split_screen_coop/
- Reference 14: https://godotassetlibrary.com/asset/QdddqG/multiplayer-input
- Reference 15: https://kidscancode.org/godot_recipes/3.x/2d/splitscreen_demo/index.html
- Reference 16: https://docs.godotengine.org/en/stable/tutorials/2d/2d_movement.html
- Reference 17: https://github.com/godotrecipes/characterbody3d_examples/blob/master/mini_tank.gd
- Reference 18: https://docs.godotengine.org/en/stable/tutorials/3d/using_transforms.html
- Reference 19: /godot-cpp/gen/src/classes/node3d.cpp
- Reference 20: /godot-cpp/src/variant/vector3.cpp
- Reference 21: /godot-cpp/include/godot_cpp/variant/vector3.hpp
- Reference 22: https://docs.godotengine.org/en/stable/tutorials/scripting/idle_and_physics_processing.html
- Reference 23: https://docs.godotengine.org/en/stable/tutorials/physics/using_character_body_2d.html
- Reference 24: https://kidscancode.org/godot_recipes/3.x/3d/3d_shooting/
- Reference 25: https://github.com/godotengine/tps-demo/blob/master/player/player.gd
- Reference 26: https://github.com/vorlac/godot-roguelite/blob/main/src/entity/projectile/projectile.cpp
- Reference 27: /godot-cpp/gen/src/classes/node3d.cpp
- Reference 28: /godot-cpp/include/godot_cpp/variant/basis.hpp
- Reference 29: https://docs.godotengine.org/en/stable/classes/class_transform3d.html#class-transform3d-method-translated

- Reference 30: https://docs.godotengine.org/en/stable/classes/class_node.html#class-node-method-get-parent
- Reference 31: https://docs.godotengine.org/en/stable/classes/class_node.html#class-node-method-add-child
- Reference 32: https://docs.godotengine.org/en/stable/engine_details/architecture/core_types.html
- Reference 33: https://github.com/godotengine/godot-cpp/blob/master/test/src/example.cpp
- Reference 34: /godot_cpp/core/math_defs.hpp
- Reference 35: https://en.cppreference.com/cpp/container/map
- Reference 36: godot-cpp/gen/include/godot_cpp/classes/mesh_instance3d.hpp
- Reference 37: godot-cpp/gen/include/godot_cpp/classes/material.hpp
- Reference 38: https://discord.com/channels/212250894228652034/342047011778068481/1487545947608322078
- Reference 39: https://discord.com/channels/212250894228652034/342047011778068481/1489038771600101457
- Reference 40: https://www.somethinglikegames.de/en/blog/2023/material-synchronization/
- Reference 41: https://docs.godotengine.org/en/4.6/getting_started/first_3d_game/07.killing_player.html
- Reference 42: https://docs.godotengine.org/en/4.6/getting_started/first_3d_game/06.jump_and_squash.html
- Reference 43: https://docs.godotengine.org/en/stable/classes/class_node.html#class-node-method-queue-free
- Reference 44: https://github.com/godotengine/godot/blob/master/core/templates/hash_set.h
- Reference 45: godot-cpp/include/godot_cpp/templates/hash_set.hpp
- Reference 46: https://stackoverflow.com/a/12863273/13097194
- Reference 47: https://github.com/kburchfiel/godot_cpp_3d_demo/tree/main/input_map_creator
- Reference 48: https://docs.godotengine.org/en/4.6/getting_started/first_2d_game/06.heads_up_display.html
- Reference 49: /godot-cpp/gen/src/variant/array.hpp
- Reference 50: /godot-cpp/include/godot_cpp/variant/variant.hpp
- Reference 51: https://docs.godotengine.org/en/stable/tutorials/scripting/pausing_games.html

- Reference 52: godot-cpp/gen/include/godot_cpp/classes/node.hpp
- Reference 53: https://docs.godotengine.org/en/stable/classes/class_timer.html
- Reference 54: /godot-cpp/gen/include/godot_cpp/classes/timer.hpp
- Reference 55: /godot-cpp/gen/include/godot_cpp/variant/dictionary.hpp
- Reference 56: https://godotforums.org/d/33822-how-to-loop-a-dictionary-in-godot-c-gdextension/3tutorial_screenshots/new_game_project.png)